



EKS and beyond..



Agenda

- Intro to Kubernetes
- Kubernetes & EKS Architecture
- Networking
- Monitoring
- Logging
- Storage
- Authentication and Security
- AutoScaling in EKS
- Hybrid your cluster (Spot)
- Best Practices

Agenda continued..

- EKS Managed Worker Nodes
- EKS with Fargate
- Elastic Container Registry (ECR)
- Q/A

What is Kubernetes?



**Open source container
management platform**



**Helps you run
containers at scale**



**Gives you primitives
for building modern
applications**

Community, contribution, choice



CLOUD NATIVE
COMPUTING FOUNDATION



kubernetes

Typical use cases

- Microservices: Java, Node.js, Go, Web Apps, etc.
- Continuous Integration and Continuous Deployment (CI/CD)
- Batch Processing and ETL jobs
- Common PaaS Stack for Application Deployment
- Legacy Application Migration to the Cloud
- Hybrid Workloads
- AI/ML
- Scale Testing
- Backend for IoT use cases

Key Stats

80%

of all containerized
applications running in
the cloud run on AWS*

82%

of all Kubernetes
applications running in
the cloud run on AWS*

150%

YoY growth of AWS
containers services

10X

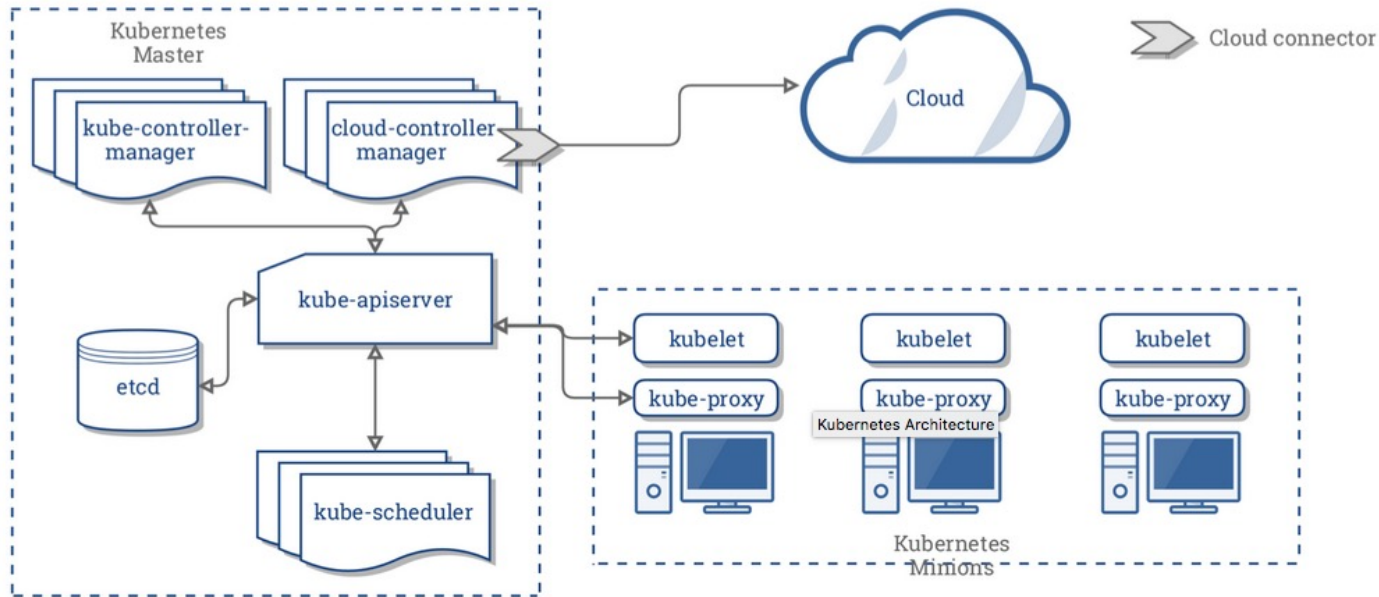
EKS usage growth
in 1 year

1.6B+

weekly image pulls
using ECR

* <https://nucleusresearch.com/research/single/guidebook-containers-and-kubernetes-on-aws/>

Kubernetes Architecture



Kubernetes Architecture



(Amazon Elastic Container Service for Kubernetes)

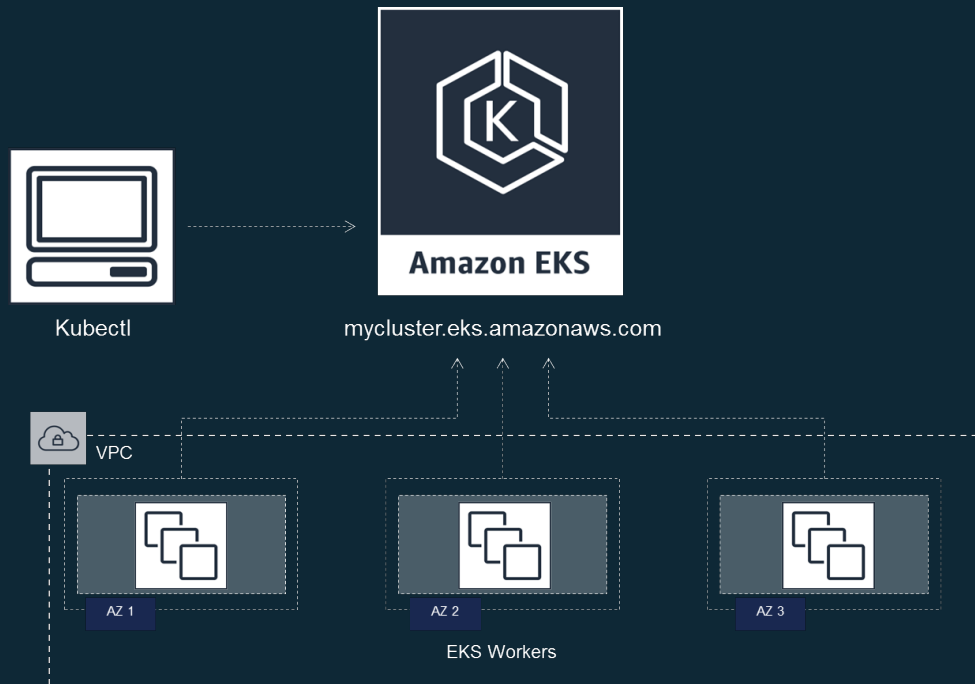
Amazon Elastic Kubernetes Service

Amazon EKS *Noun.* \ 'a-mə- ,zän ,i: ,keɪ ,ɛs \

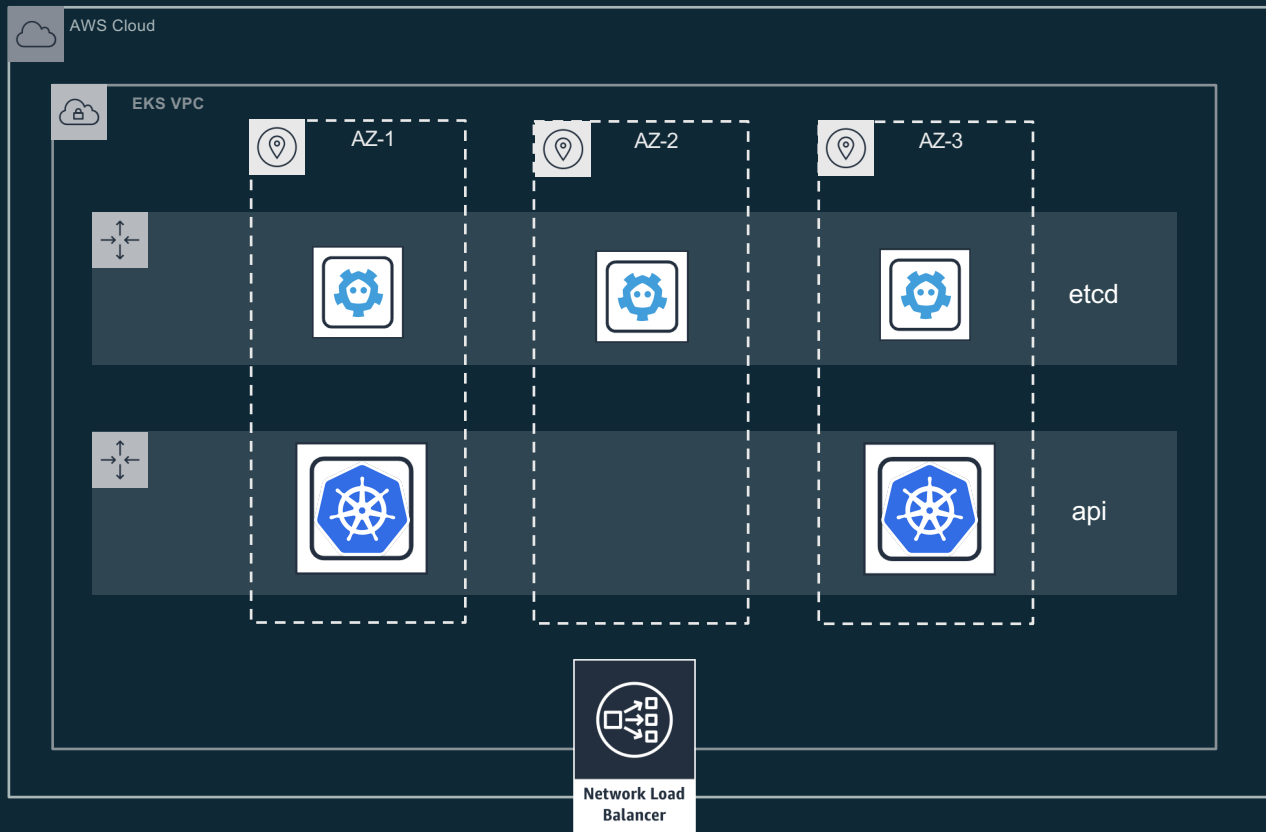
Production-ready, highly-available, managed Kubernetes service for AWS.

1. Platform for enterprises to run production-grade workloads
2. Provides a native and upstream Kubernetes experience
3. AWS integrations are seamless and eliminate undifferentiated heavy lifting
4. EKS team actively contributes to the Kubernetes community

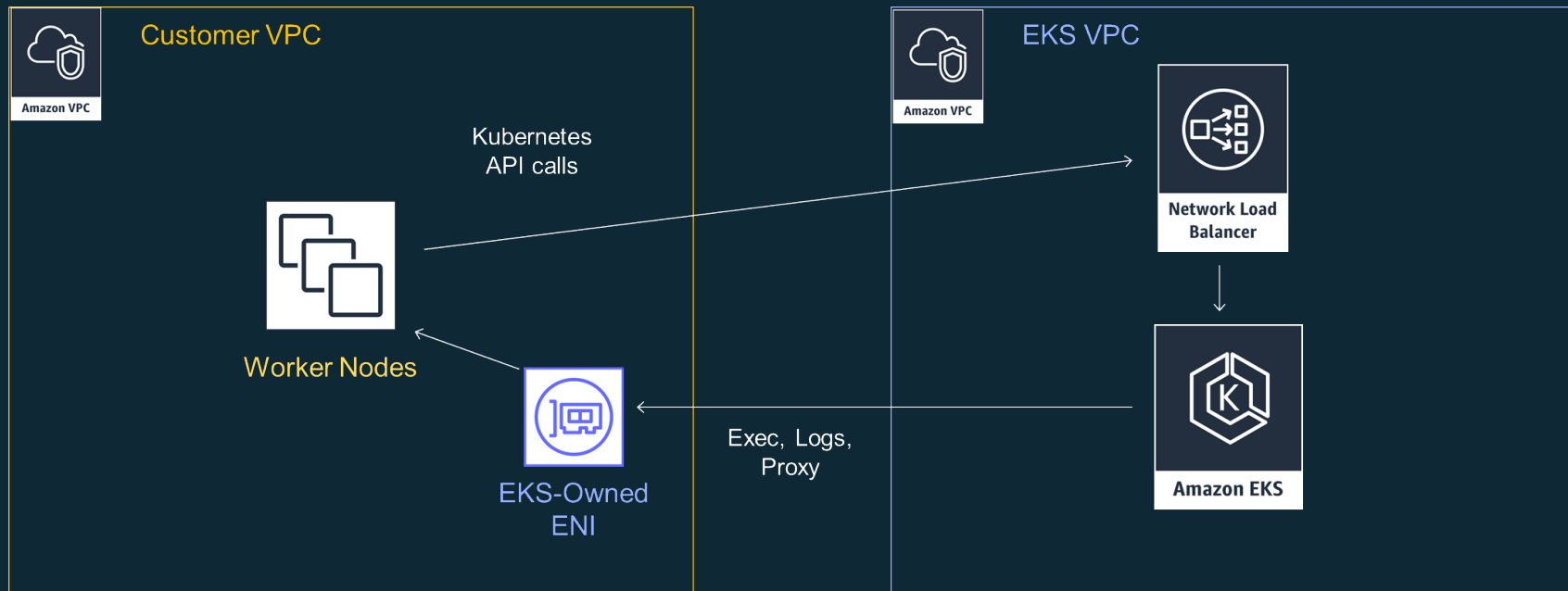
EKS Overview



EKS Architecture



EKS Architecture



Objects

Kubernetes objects are entities that are used to represent the state of the cluster (SPEC).

An object is a “record of intent” – once created, the cluster does its best to ensure it exists as defined. This is known as the cluster’s “desired state.”

A desired state can describe:

- What pods (containers) are running, and on which nodes
- IP endpoints that map to a logical group of containers
- How many replicas of a container are running
- And much more...

Object...Cntd

Pod - A thin wrapper around one or more containers

DaemonSet - Implements a single instance of a pod on a worker node

Deployment - Details how to roll out (or roll back) across versions of your application

ReplicaSet - Ensures a defined number of pods are always running

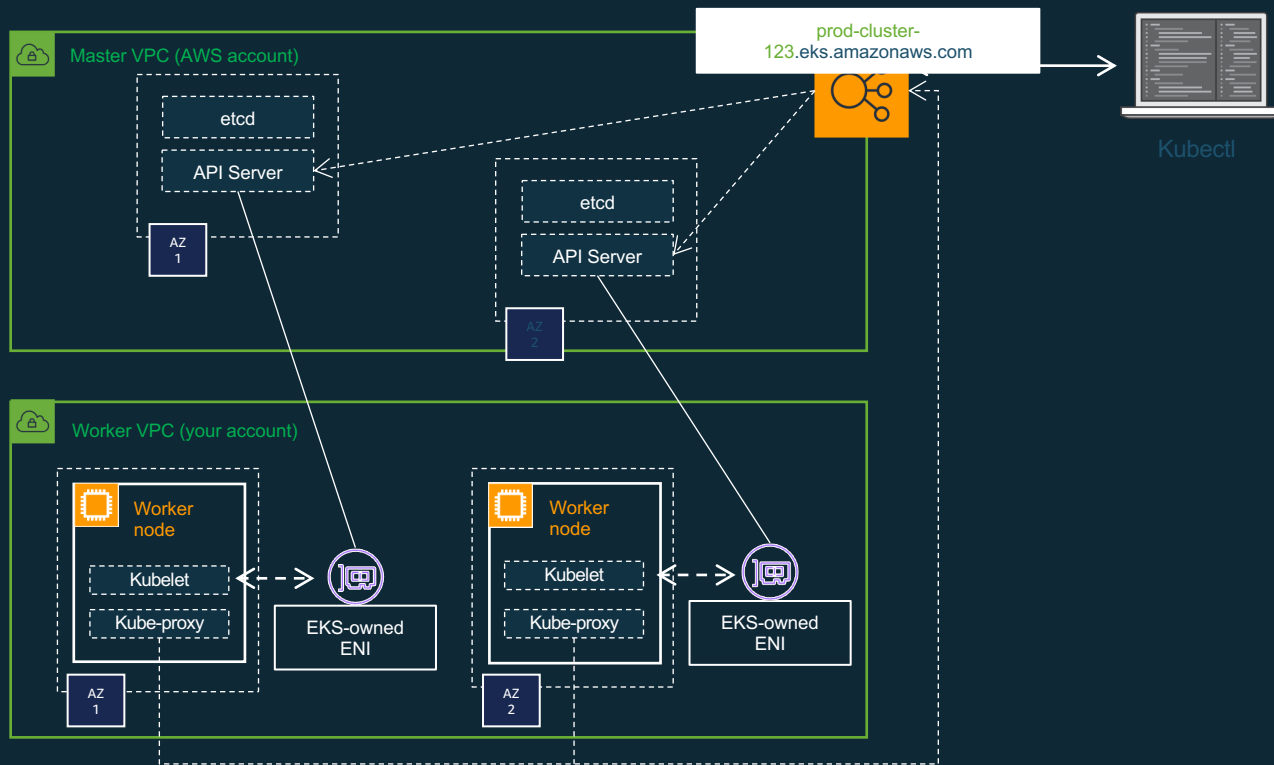
Job - Ensures a pod properly runs to completion

Service - Maps a fixed IP address to a logical group of pods

Label - Key/Value pairs used for association and filtering

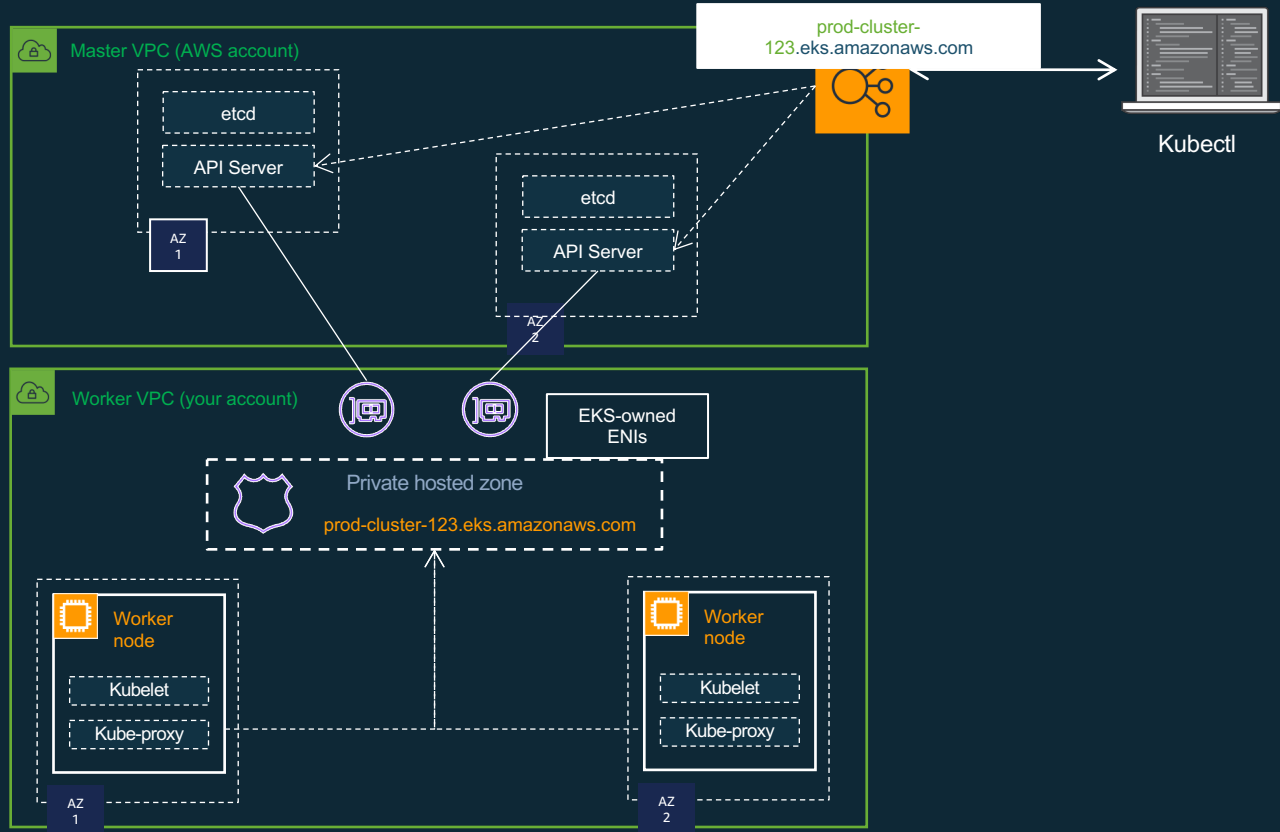
Cloud Native Security: API-server Endpoint Access Control

`public == true`
`private == false`



API-server Endpoint Access Control

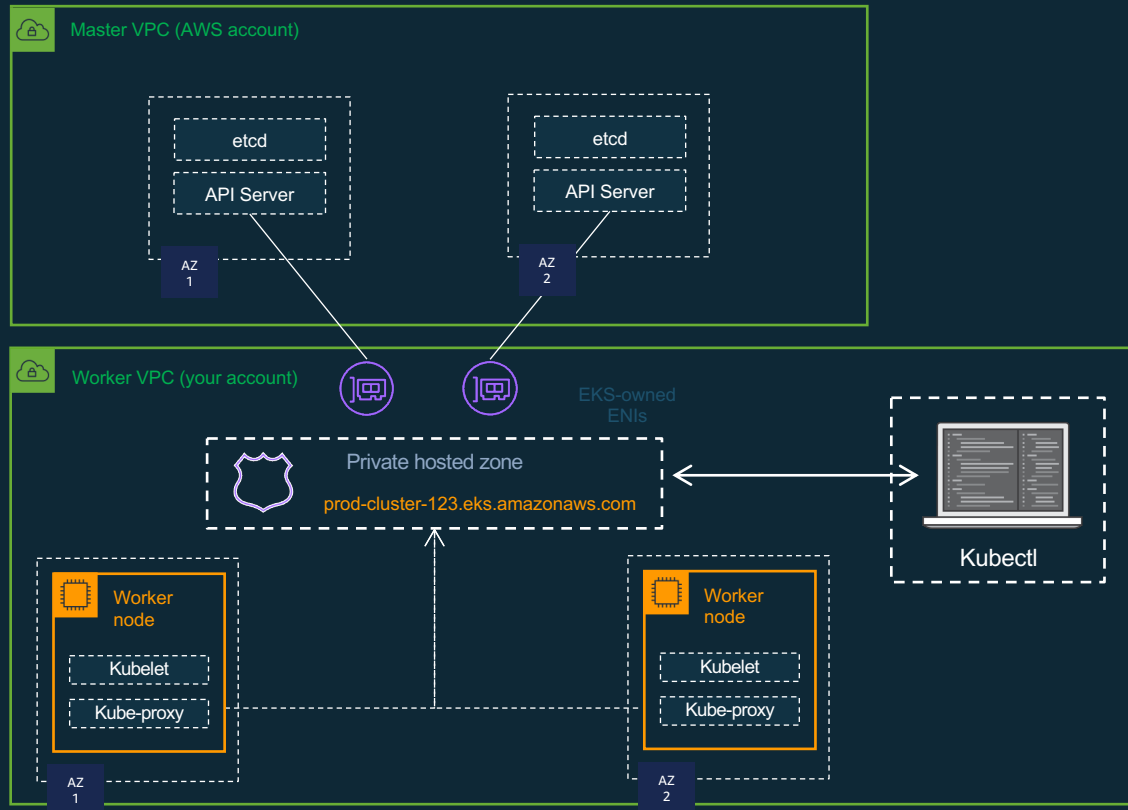
public == true
private == true



API-server Endpoint Access Control

`public == false`

`private == true`



EKS is Kubernetes Certified



You can use existing tooling and plugins from partners and the Kubernetes community

Flexibility focused

 Container-level networking

 Managed etcd and masters

 Integrated with AWS platform

 Kubernetes APIs

 Kubernetes ecosystem

 Timely upgrades

 Highly available masters

 100% upstream Kubernetes

Amazon EKS is ready for sensitive and regulated workloads

HIPAA-eligible

ISO 9001, 27001, 27017, 27018

PCI DSS

SOC 1,2,3

Rich partner ecosystem

Foundation

CANONICAL

RANCHER



DevOps



GitLab

ATLASSIAN

Monitoring & Logging



Security



Networking



TIGERA
CLOUD NETWORKS, SECURED



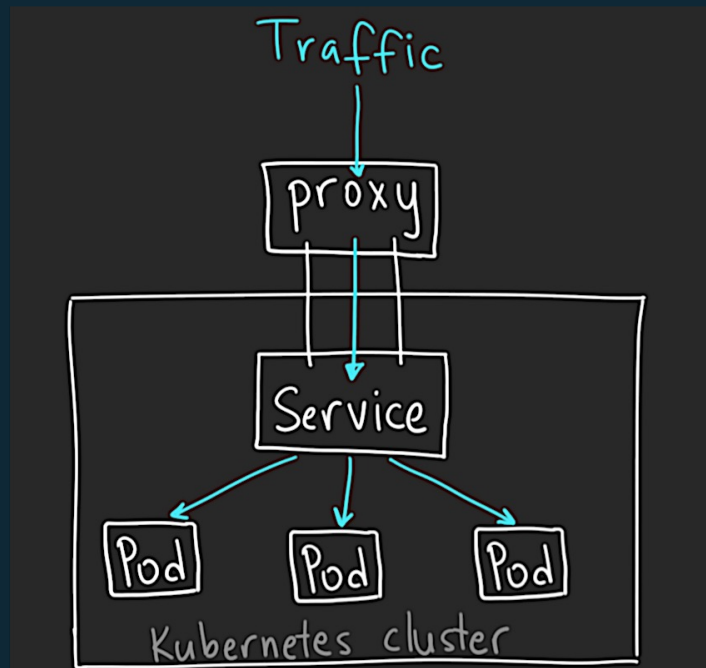
Kubernetes Networking

The three rules of Kubernetes networking...

- All the pods can communicate with each other directly without NAT
- All the nodes can communicate with all pods (and vice versa) without NAT
- The IP that a pod sees itself as is the same IP that others see it as

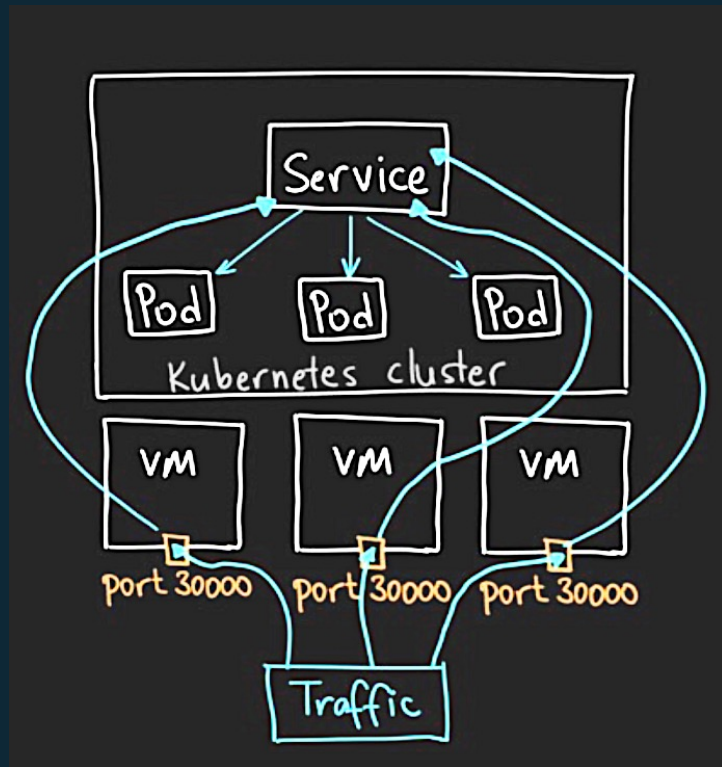
Kubernetes ServiceType: ClusterIP

- Exposes the service on a cluster-internal IP
- Only reachable from within the cluster
- Access possible via kube-proxy
- Useful for debugging services, connecting from your laptop or displaying internal dashboards



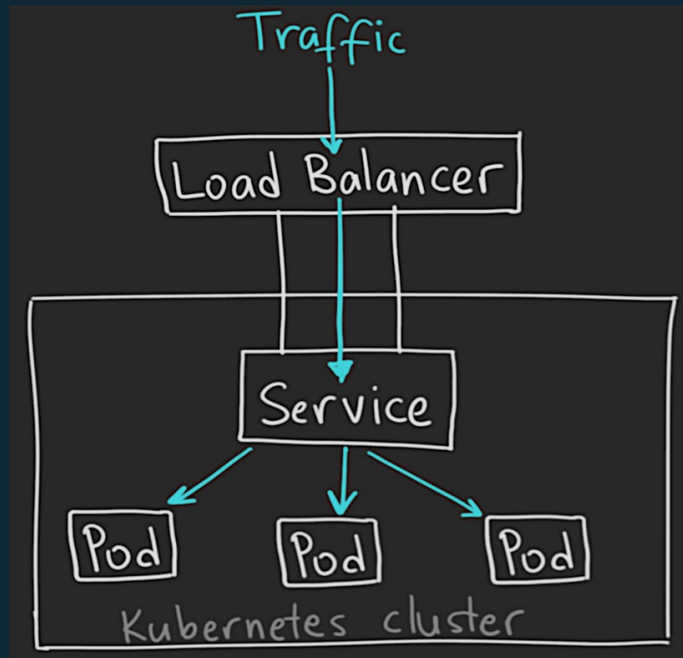
Kubernetes ServiceType: NodePort

- Exposes the service on each Node's IP at a static port.
- Routes to a ClusterIP service, which is automatically created.
- from outside the cluster:
<NodeIP>:<NodePort>
- 1 service per port
- Uses ports 30000-32767



Kubernetes ServiceType: LoadBalancer

- Exposes the service **externally** using a cloud provider's load balancer.
- NodePort and ClusterIP services (to which LB will route) automatically created.
- Each service exposed with a LoadBalancer (ELB or NLB) will get its own IP address
- Exposes L4 (TCP) or L7 (HTTP) services



Service load balancer: NLB

apiVersion: v1

kind: Service

metadata:

name: nginx

namespace: default

labels:

app: nginx

annotations:

[service.beta.kubernetes.io/aws-load-balancer-type: "nlb"](https://kubernetes.io/docs/concepts/services-networking/service/#load-balancer)

spec:

externalTrafficPolicy: Local

ports:

- name: http

port: 80

protocol: TCP

targetPort: 80

selector:

app: nginx

type: LoadBalancer

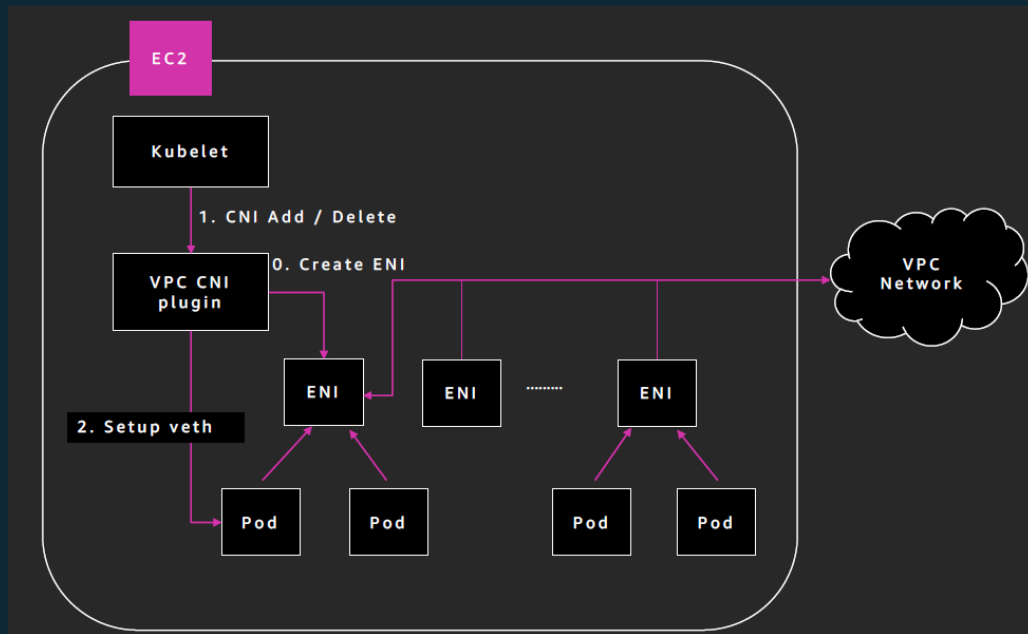
Kubernetes ServiceType: ExternalName

- Maps: service => CNAME(*externalName* field)
- No proxying
- Accessing my-service works in the same way as other Services
- redirection happens at the DNS level (rather than via proxying or forwarding)

```
kind: Service
apiVersion: v1
metadata:
  name: my-service
  namespace: prod
spec:
  type: ExternalName
  externalName:
    my.database.example.com
```

AWS VPC CNI plugin - configurability

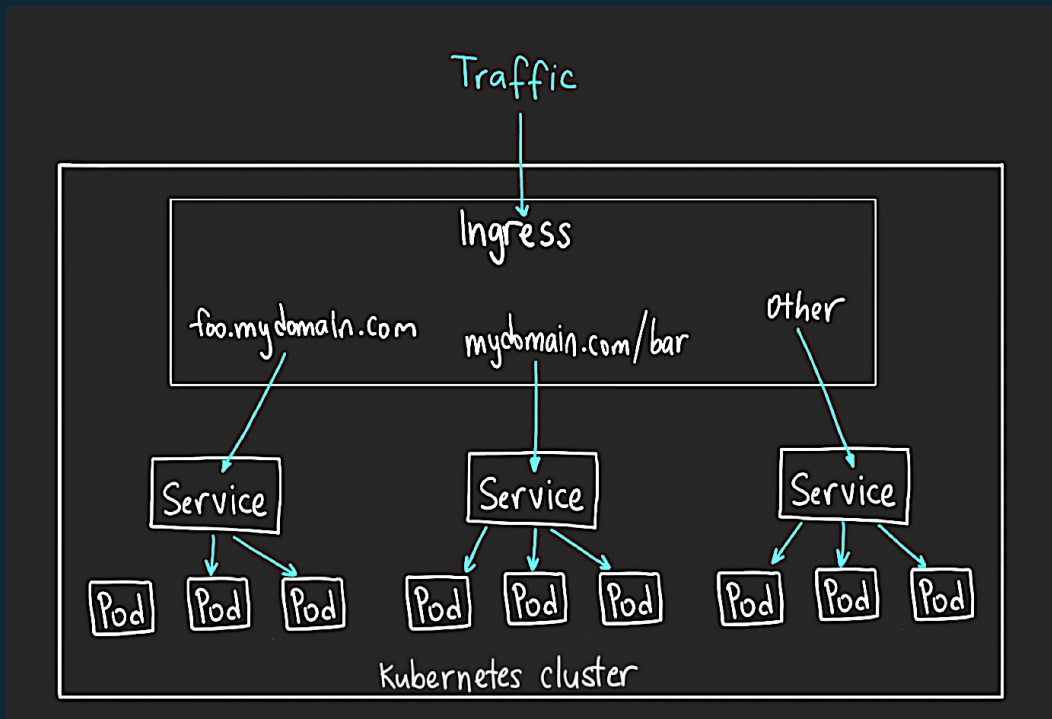
- Thin layer, no overhead
- Give K8s pods native IP addresses in the VPC
- Multiple ENI per Amazon EC2, multiple pods per ENI



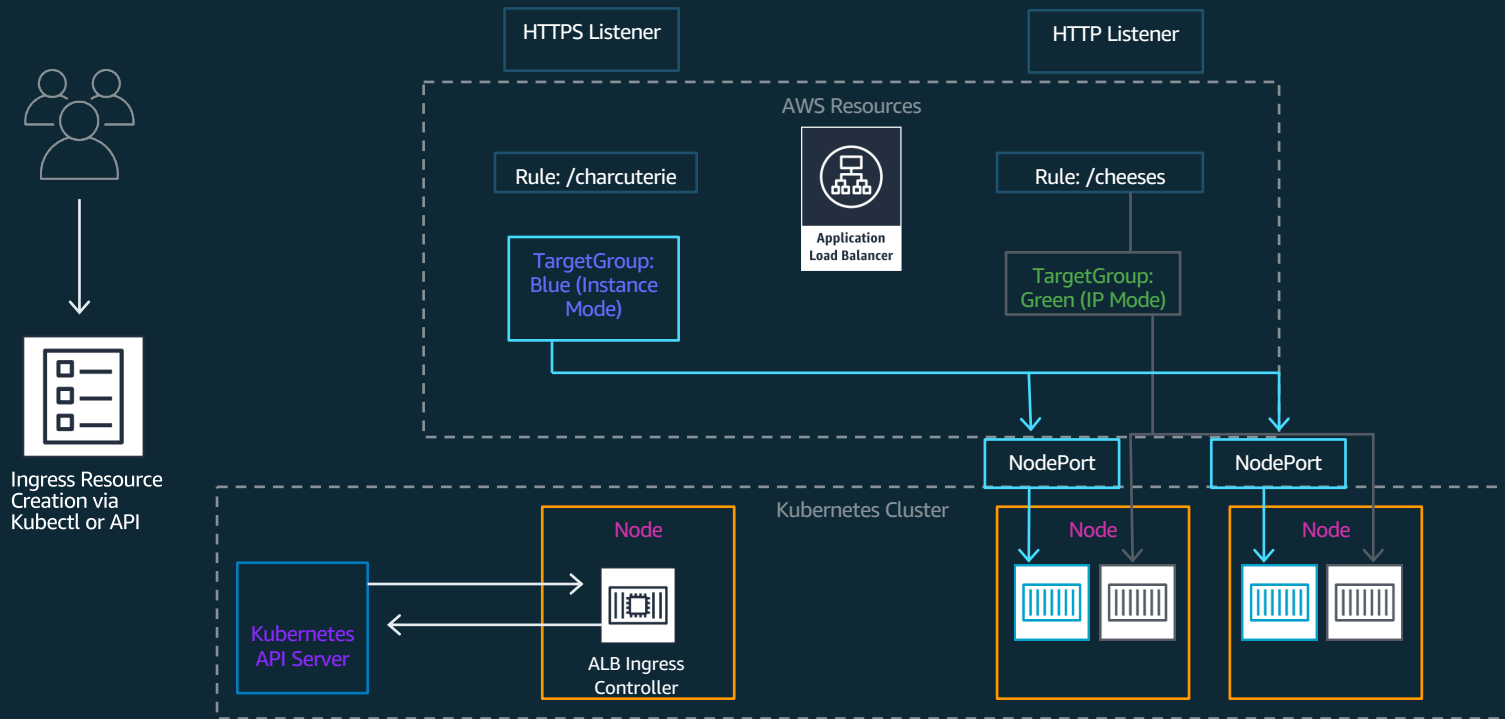
<https://github.com/aws/amazon-vpc-cni-k8s>

Kubernetes Ingress Object

- exposes HTTP/HTTPS routes to services within the cluster
- Many implementations: ALB, Nginx, F5, HAProxy etc
- Default Service Type: ClusterIP

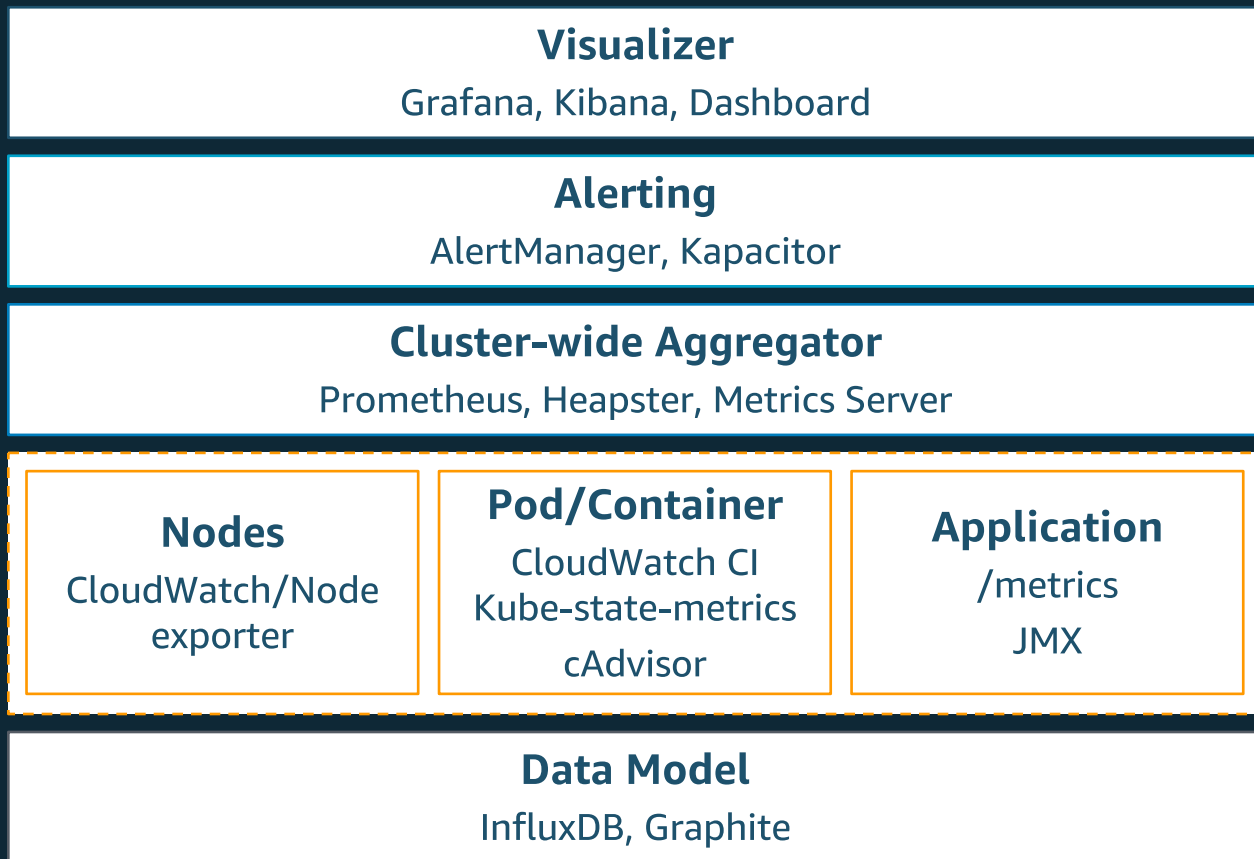


ALB Ingress Controller



Monitoring in k8s | EKS

How to monitor k8s cluster



Amazon CloudWatch

Gives you complete visibility into your cloud resources and applications so you can monitor, troubleshoot and remediate issues.



CloudWatch Container Insights

A fully managed observability service for monitoring, troubleshooting and alarming on your containerized applications and microservices.

- ✓ Collects, aggregates, and summarizes
- ✓ Reliable, secure metrics and logs collection
 - ✓ Automated dashboards and analysis
- ✓ Observability experience across metrics, logs, traces
 - ✓ Ad hoc analytics

Images on DockerHub

Performance Metrics – CloudWatch Agent

<https://hub.docker.com/r/amazon/cloudwatch-agent>

- Tag: latest

Logs – Fluentd

<https://hub.docker.com/r/fluent/fluentd-kubernetes-daemonset>

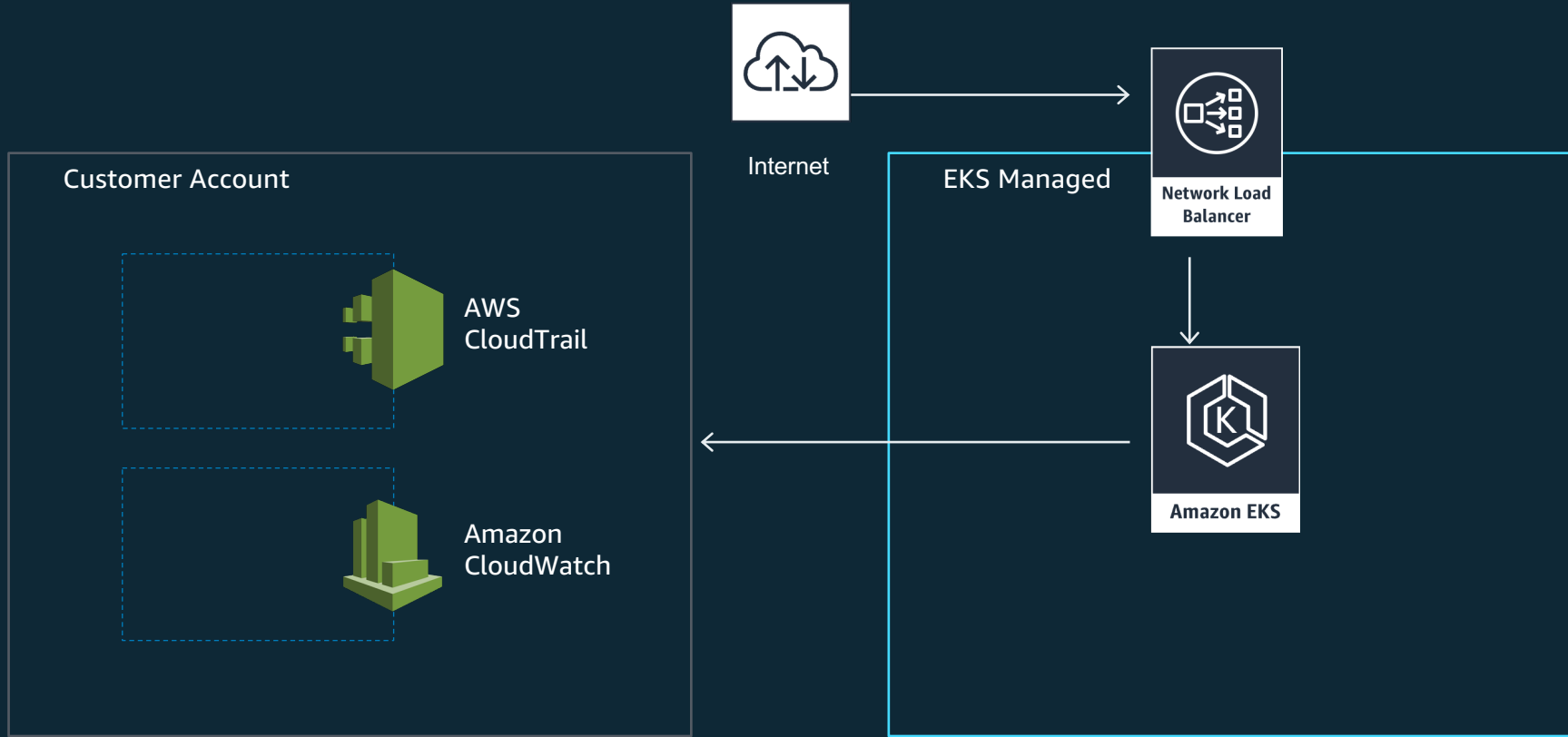
- Tag: v1.3.3-debian-cloudwatch-1.4

Container Insights

1. **A fully managed**, AWS native observability service providing automated summary and analysis of compute capacity.
2. **Reliable and secure collection** of application logs with built in analytics capabilities.
3. **Prebuilt visualization** to summarize cluster and node errors.
4. **Application / Microservice Tracing** - Troubleshoot and debug application & microservice.

Logging in k8s | EKS

EKS Logging



Cloudwatch and EKS integration: Logging

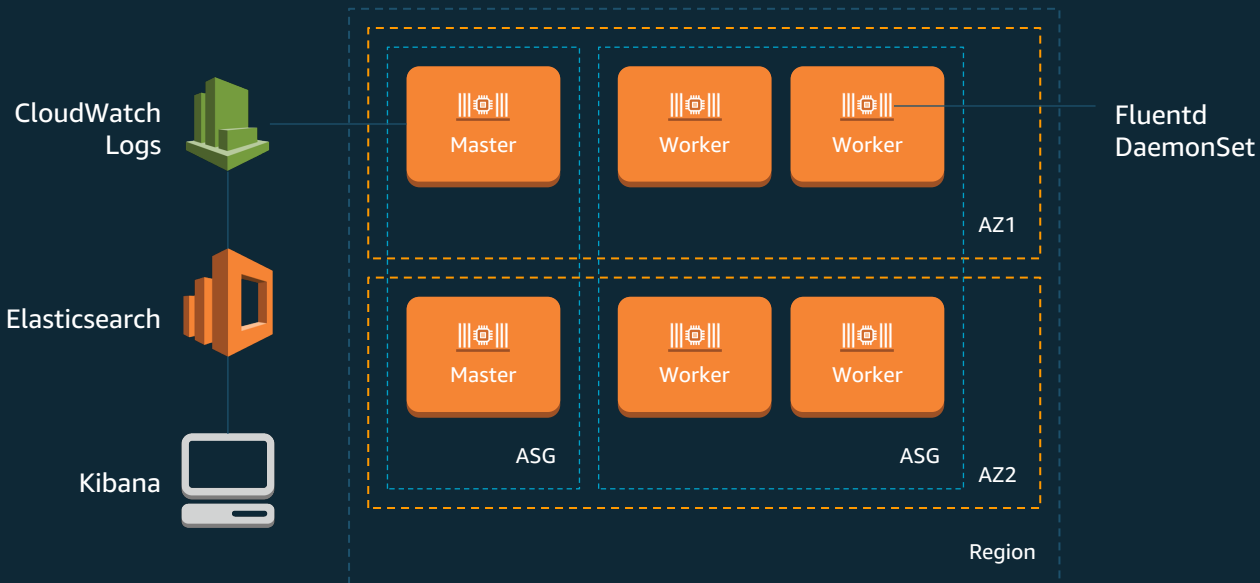
- EKS Control plane logging
 - Kubernetes API server component logs (api)
 - Audit
 - Authenticator
 - Controller manager
 - Scheduler
- Logging to CloudWatch

<https://eksworkshop.com/logging/>

EKS Logging

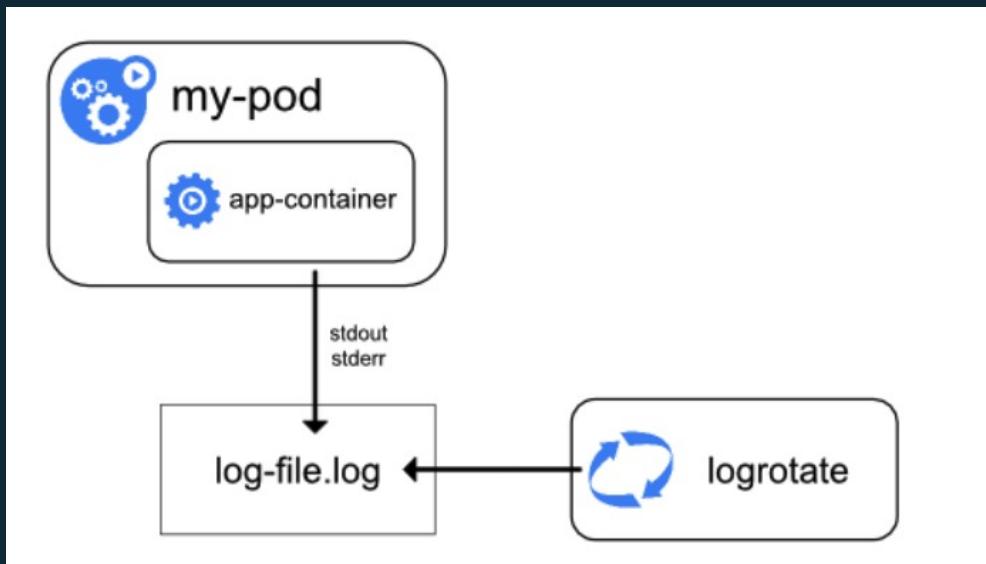
Kubectl logs

Elasticsearch (index),
Fluentd (store), and
Kibana (visualize)



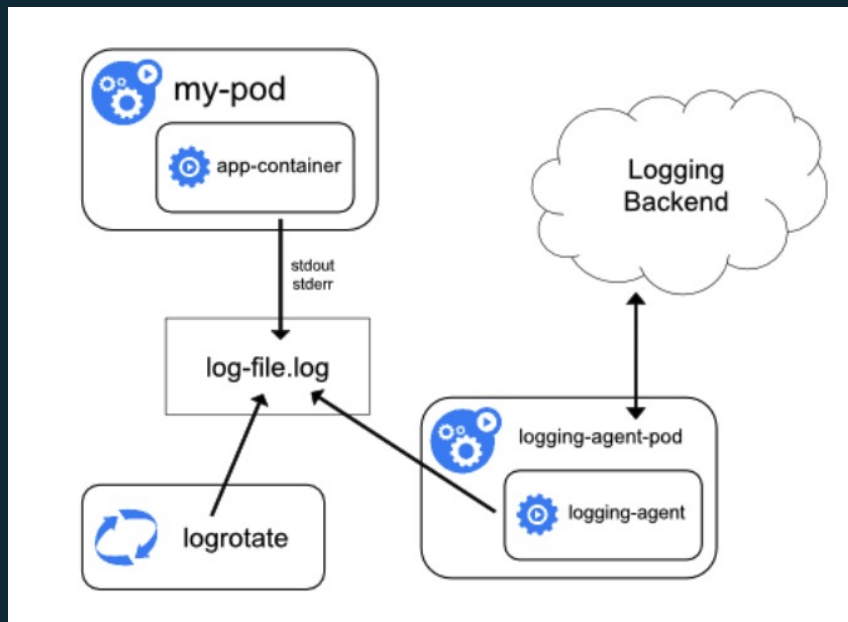
Container logging architecture

- Logging at the node level



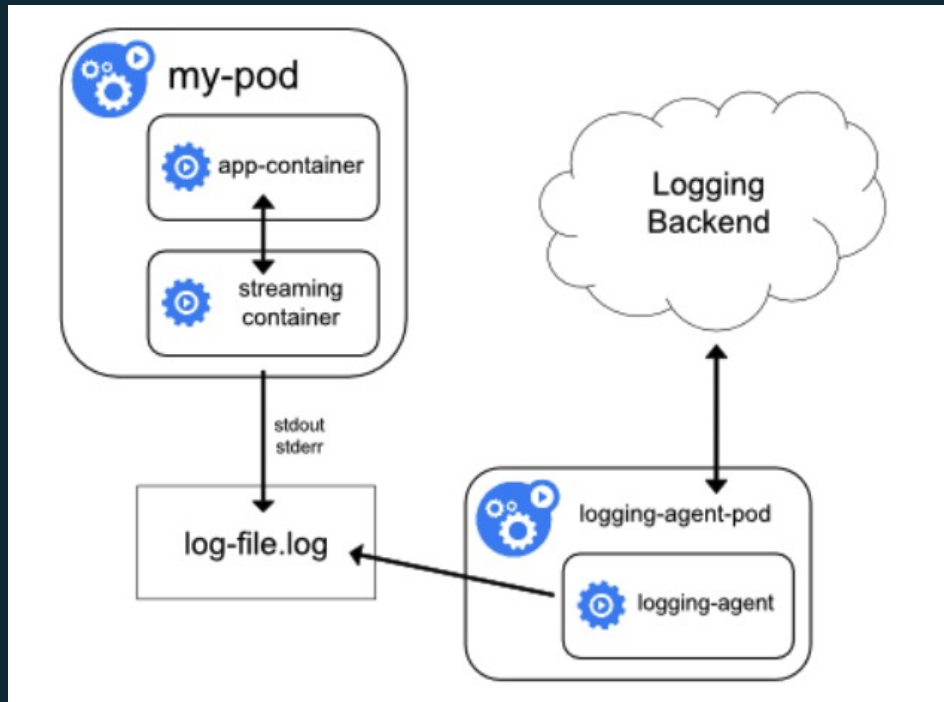
Container logging architecture

- Using a node logging agent



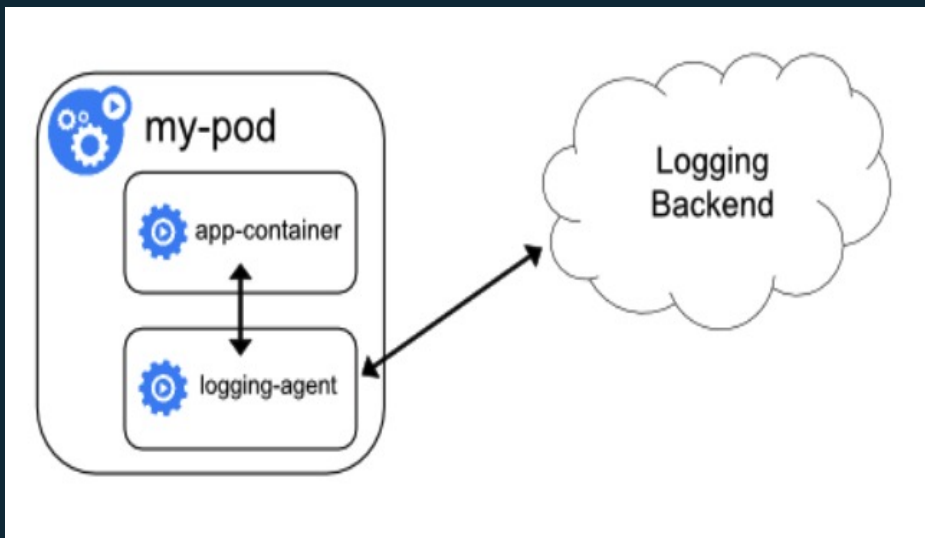
Container logging architecture

- Streaming sidecar container



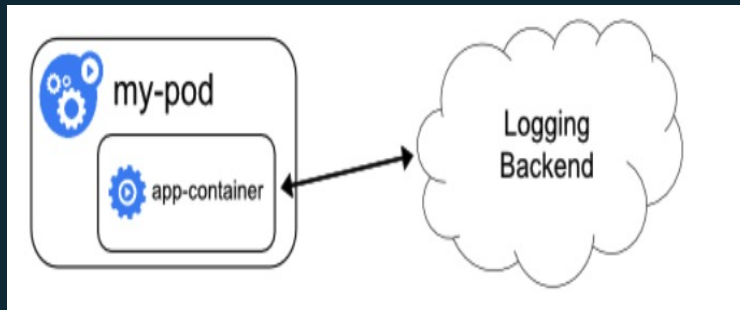
Container logging architecture

- Sidecar container with a logging agent



Container logging architecture

- **Exposing logs directly from the application**



Monitoring EKS API calls in Cloudtrail

- CreateCluster
- DeleteCluster
- UpdateClusterConfig
- UpdateClusterVersion
- DescribeCluster, DescribeUpdate, ListClusters, ListUpdates

Note: CloudTrail would not capture the events such as creation / deletion of deployments, services etc. Enable EKS control plane logging to get audit and diagnostic logs from EKS control plane.

Storage in k8s | EKS

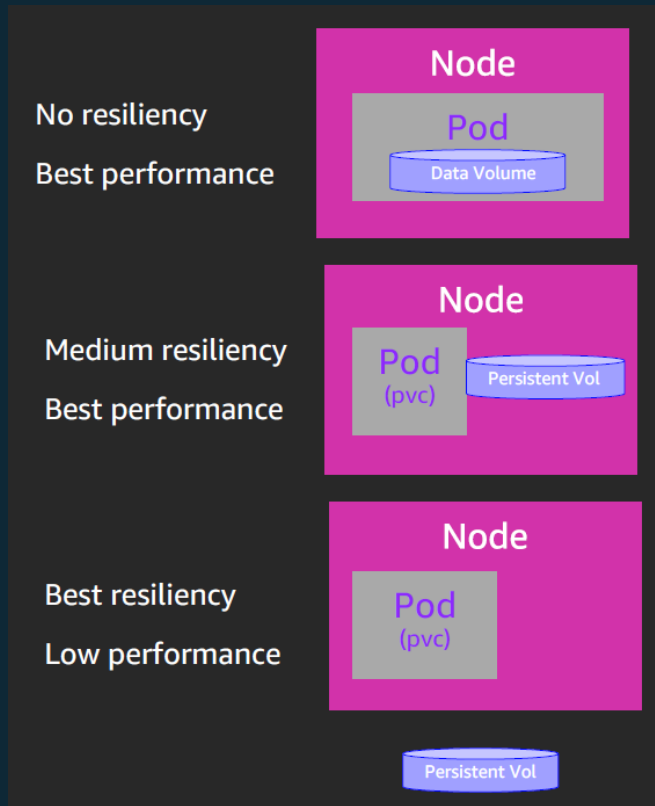
k8s Storage in AWS

Persistent Volume (PV)

- Persistent storage survives pod restarts
- HostPathPV
- Local storage SSD/NVMe devices
- PV are attached via PV Claims

PV Claims (PVC)

- Dynamic
- Abstraction to underlying storage



Container Storage Interface (CSI)

A flexible standard for orchestration
and storage provider connections



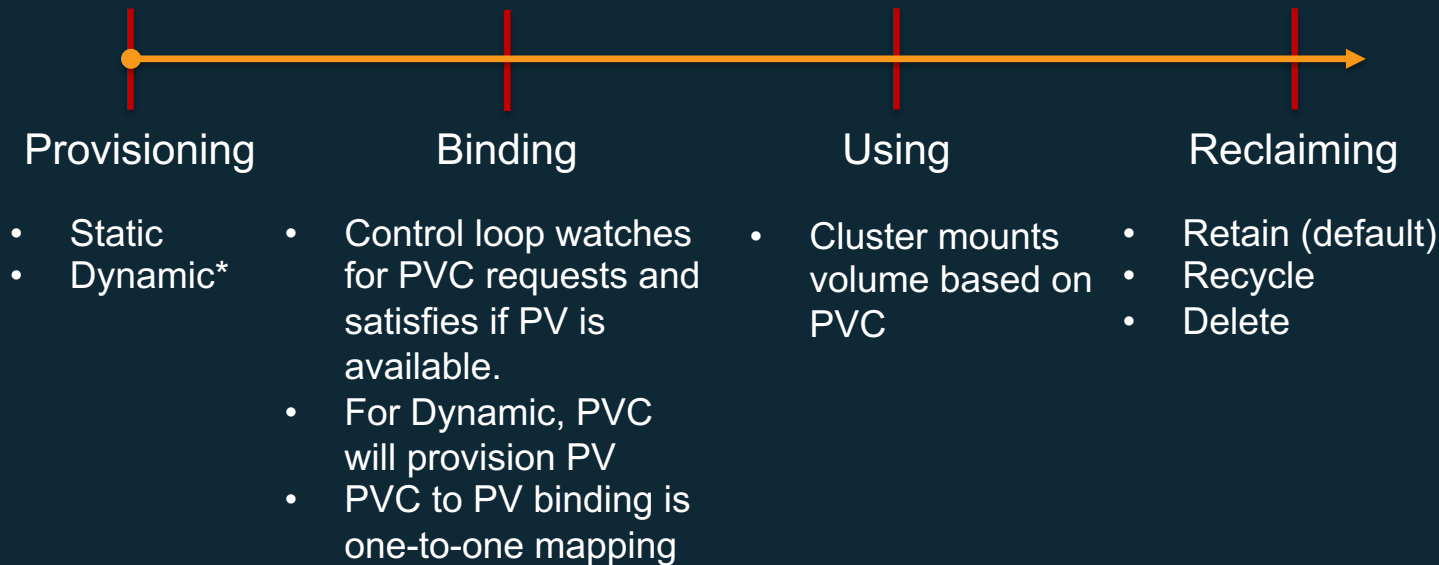
We support the CSI standard through following drivers:

Amazon Elastic Block Store: AWS EBS CSI Driver

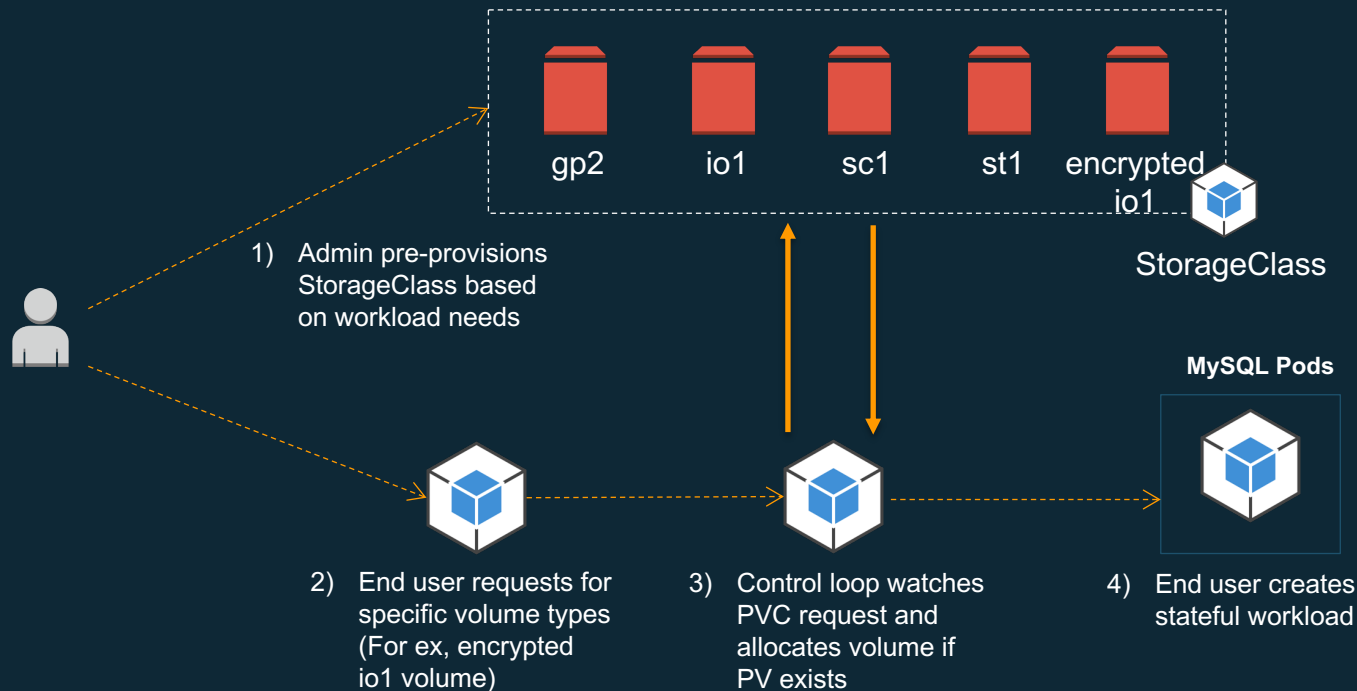
Amazon Elastic File System: AWS EFS CSI Driver

Amazon FSx for Lustre: AWS FSx CSI Driver

Storage volume lifecycle



What if I need specific volume type?



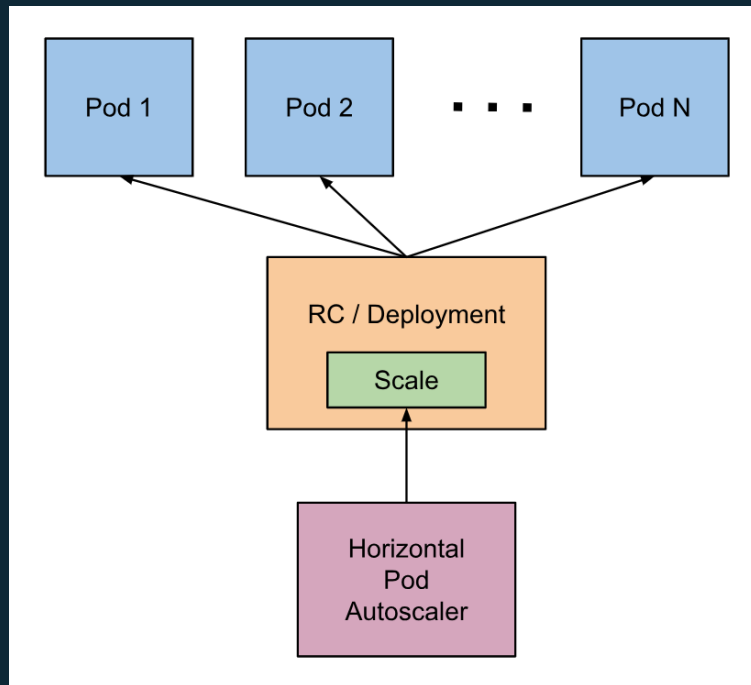
AutoScaling in k8s | EKS

Auto Scaling with Amazon EKS

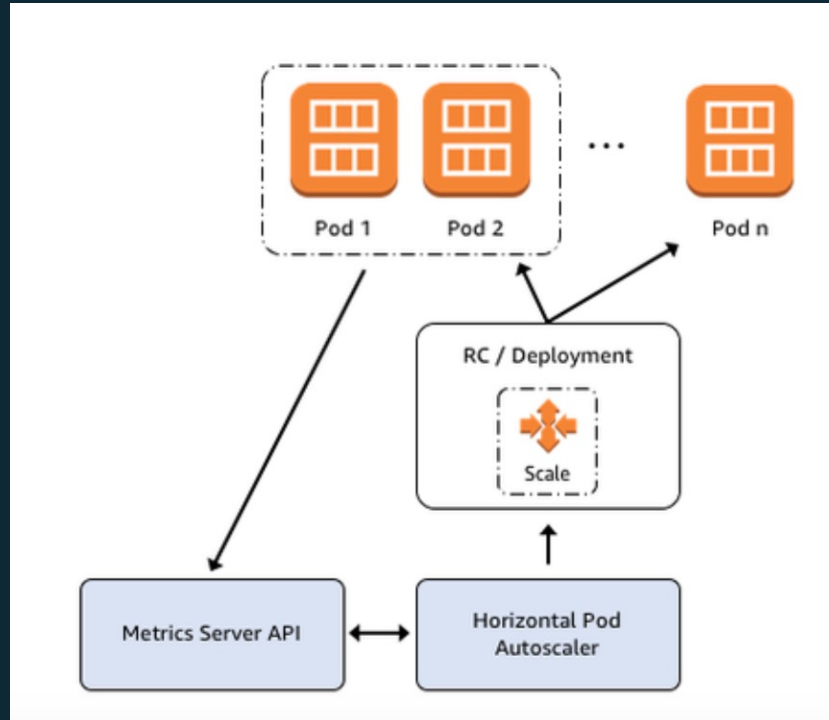
Two dimensions to scaling

- Amazon EC2 instance through cluster-autoscaler
 - scale out Amazon EC2 Spot Instances
- Pods through horizontal pod autoscaler (HPA)
 - scale out via changing pod replica count

HPA (Horizontal Pod Autoscaler)



The “how” part..

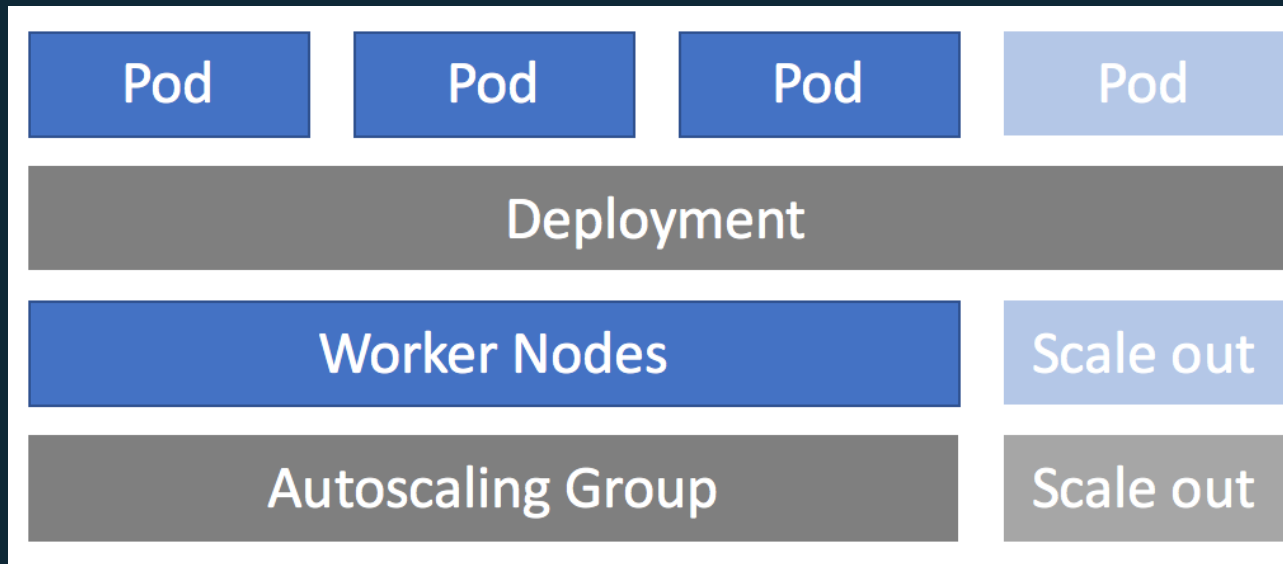


HPA in Action

```
soconnor:~/environment/metrics-server (master) $ kubectl get hpa -w
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
php-apache	Deployment/php-apache	1%/50%	1	10	1	40s
php-apache	Deployment/php-apache	347%/50%	1	10	1	1m
php-apache	Deployment/php-apache	347%/50%	1	10	4	1m
php-apache	Deployment/php-apache	116%/50%	1	10	4	2m
php-apache	Deployment/php-apache	116%/50%	1	10	4	2m
php-apache	Deployment/php-apache	123%/50%	1	10	4	3m
php-apache	Deployment/php-apache	123%/50%	1	10	4	3m
php-apache	Deployment/php-apache	129%/50%	1	10	4	4m
php-apache	Deployment/php-apache	129%/50%	1	10	4	4m
php-apache	Deployment/php-apache	122%/50%	1	10	4	5m
php-apache	Deployment/php-apache	122%/50%	1	10	8	5m
php-apache	Deployment/php-apache	59%/50%	1	10	8	6m
php-apache	Deployment/php-apache	59%/50%	1	10	8	6m
php-apache	Deployment/php-apache	58%/50%	1	10	8	7m
php-apache	Deployment/php-apache	58%/50%	1	10	8	7m
php-apache	Deployment/php-apache	57%/50%	1	10	8	8m
php-apache	Deployment/php-apache	57%/50%	1	10	8	8m
php-apache	Deployment/php-apache	60%/50%	1	10	8	9m
php-apache	Deployment/php-apache	60%/50%	1	10	10	9m
php-apache	Deployment/php-apache	47%/50%	1	10	10	10m
php-apache	Deployment/php-apache	47%/50%	1	10	10	10m

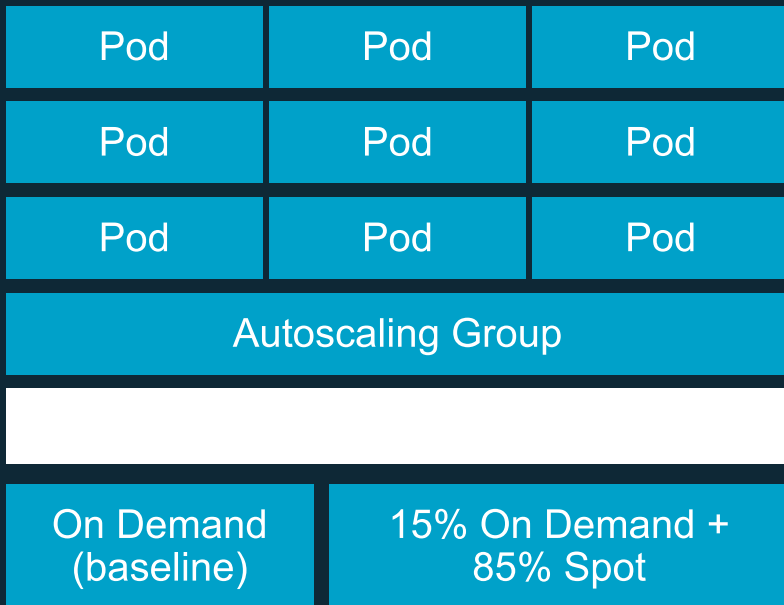
CA (Cluster Autoscaler)



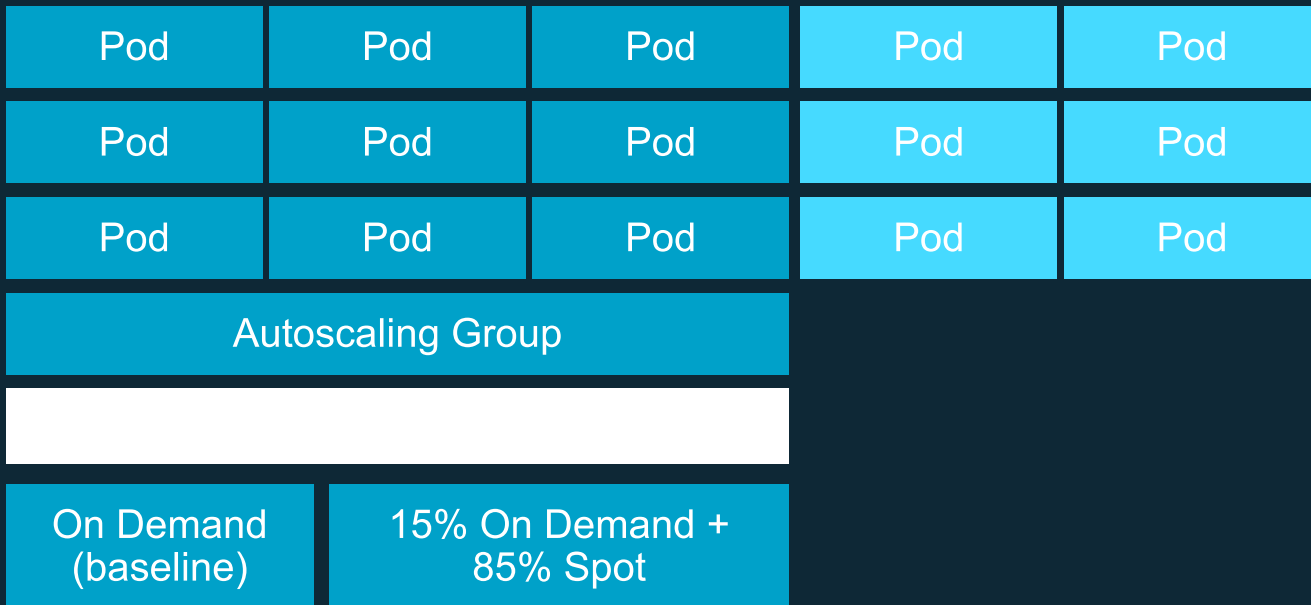
<https://github.com/kubernetes/autoscaler/tree/master/cluster-autoscaler>

Hybrid Your Cluster K8 | EKS

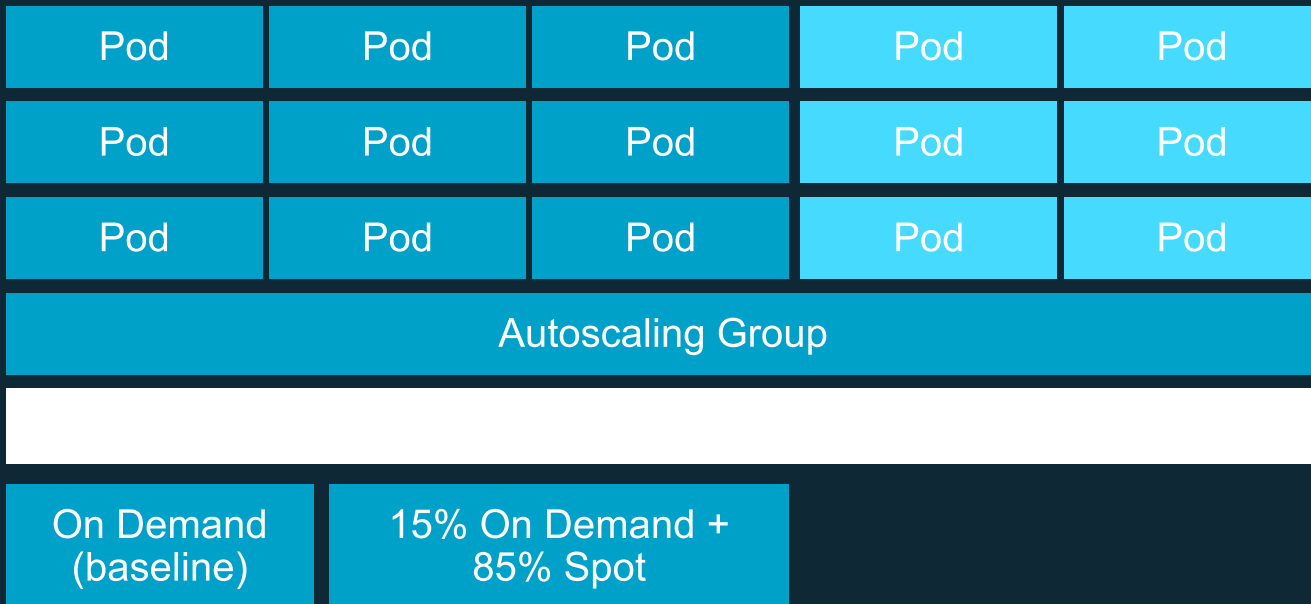
Hybrid Your Cluster



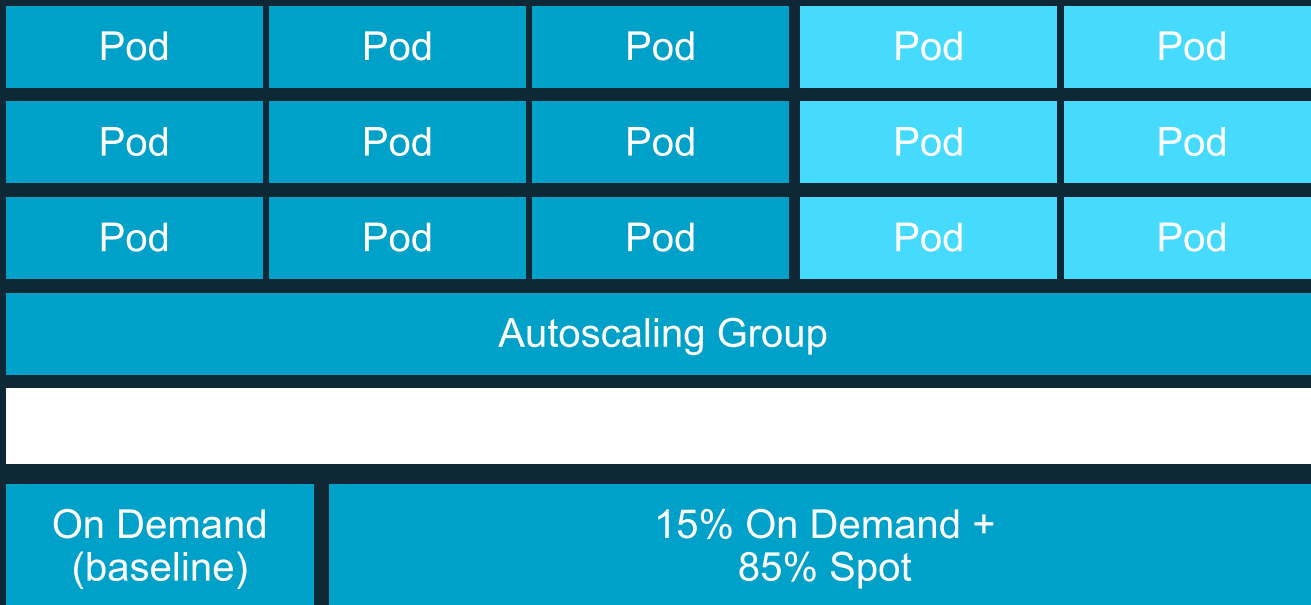
HPA Scaling Out



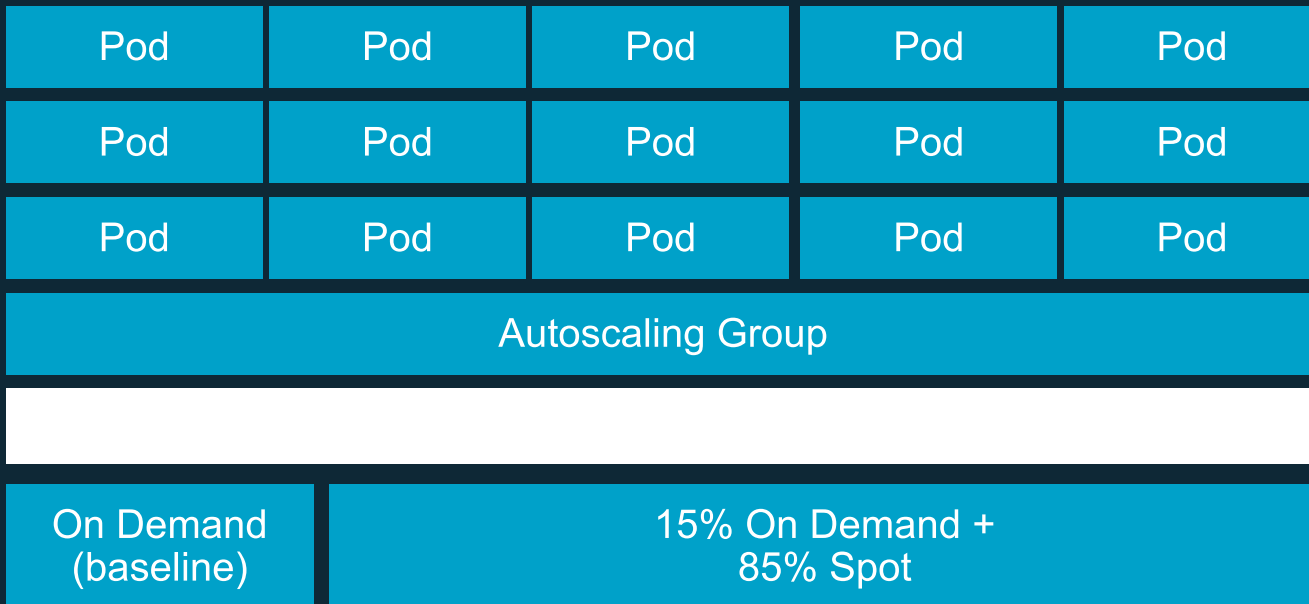
Cluster Autoscaling



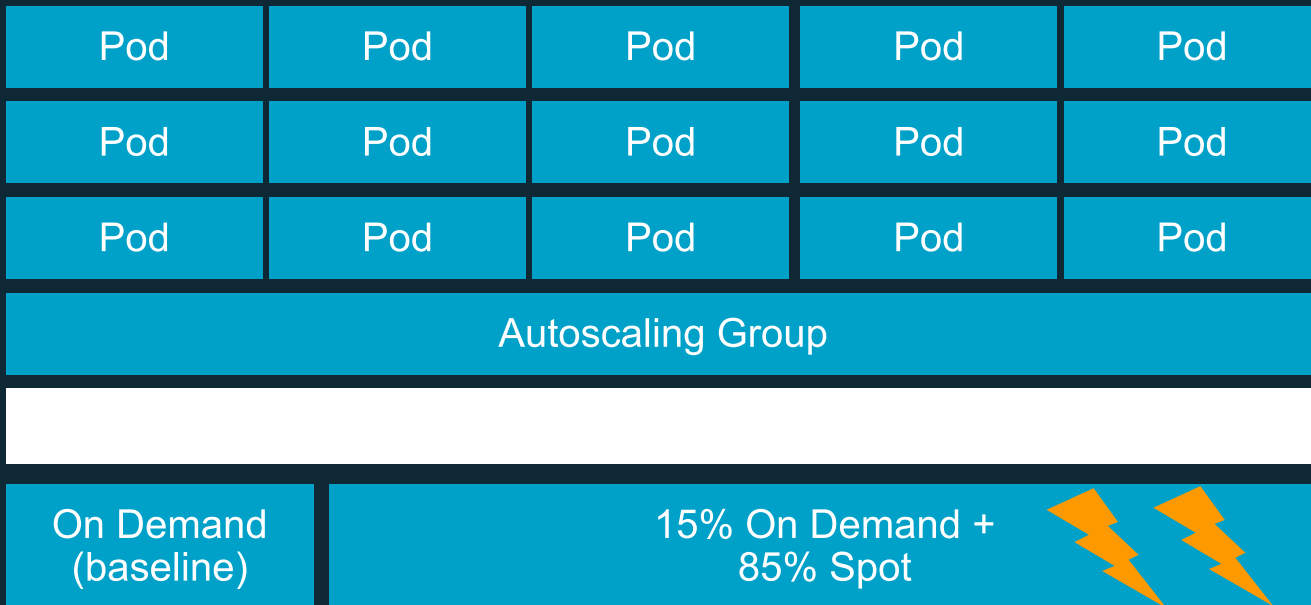
Cluster Autoscaling



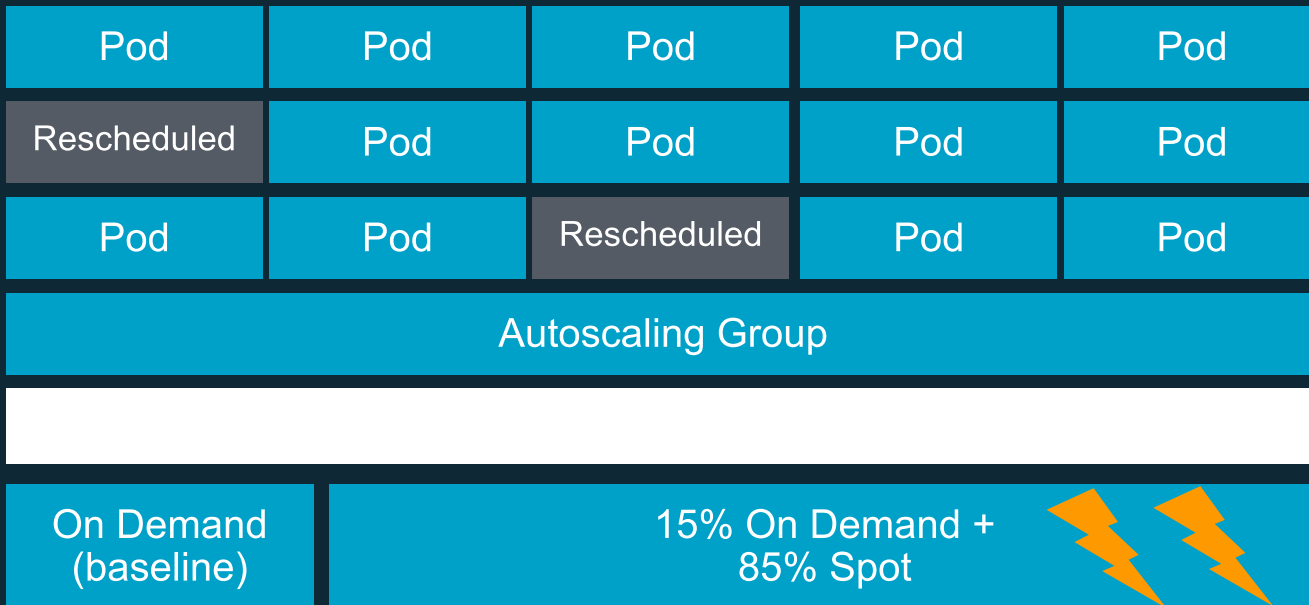
Pod and Cluster both Scaled



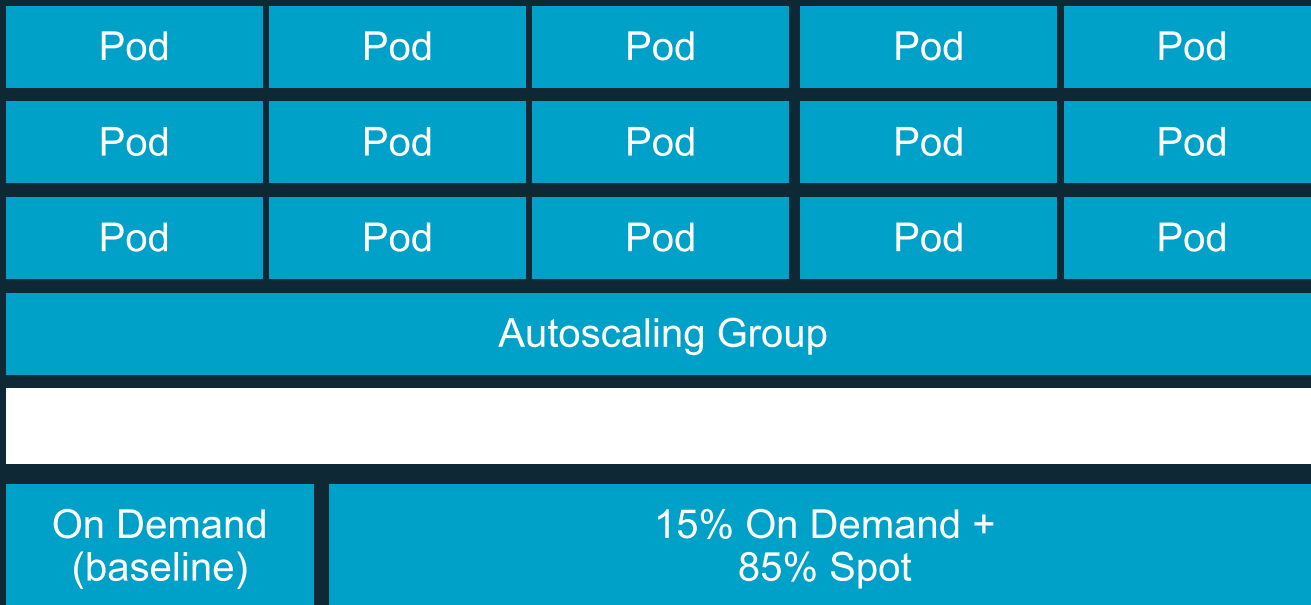
Spot Termination Handling



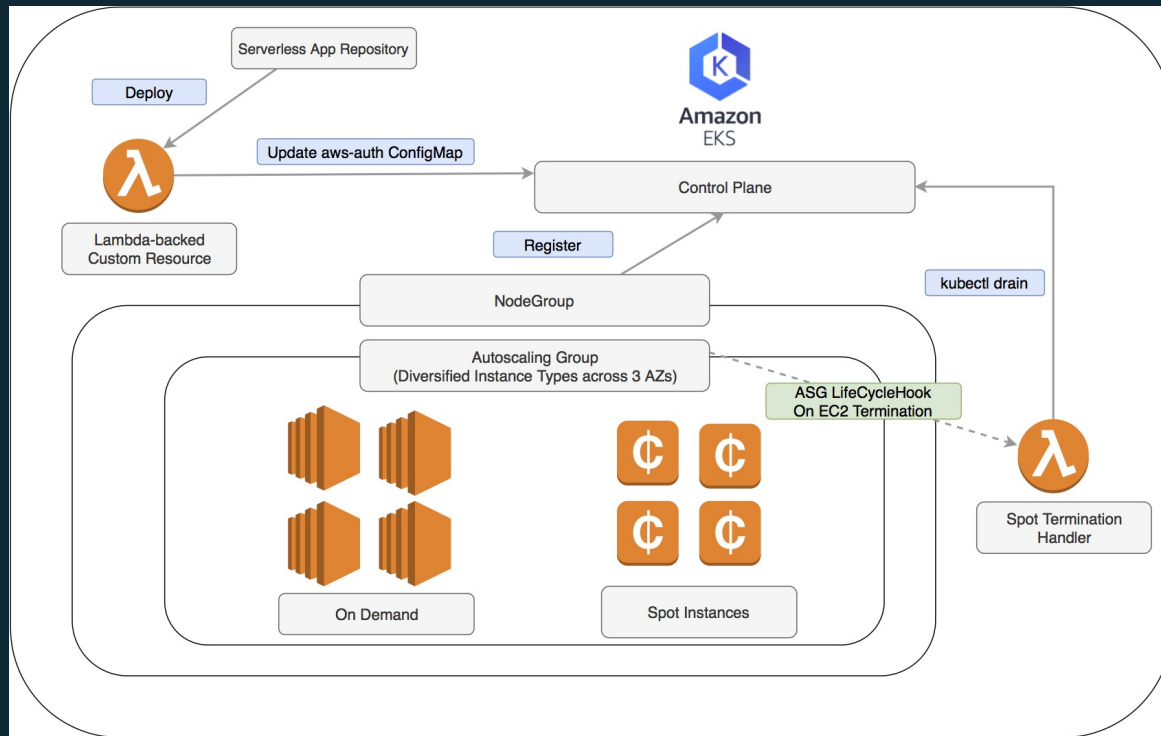
Node Draining and Pod Rescheduled



ASG Maintaining the Desired Capacity



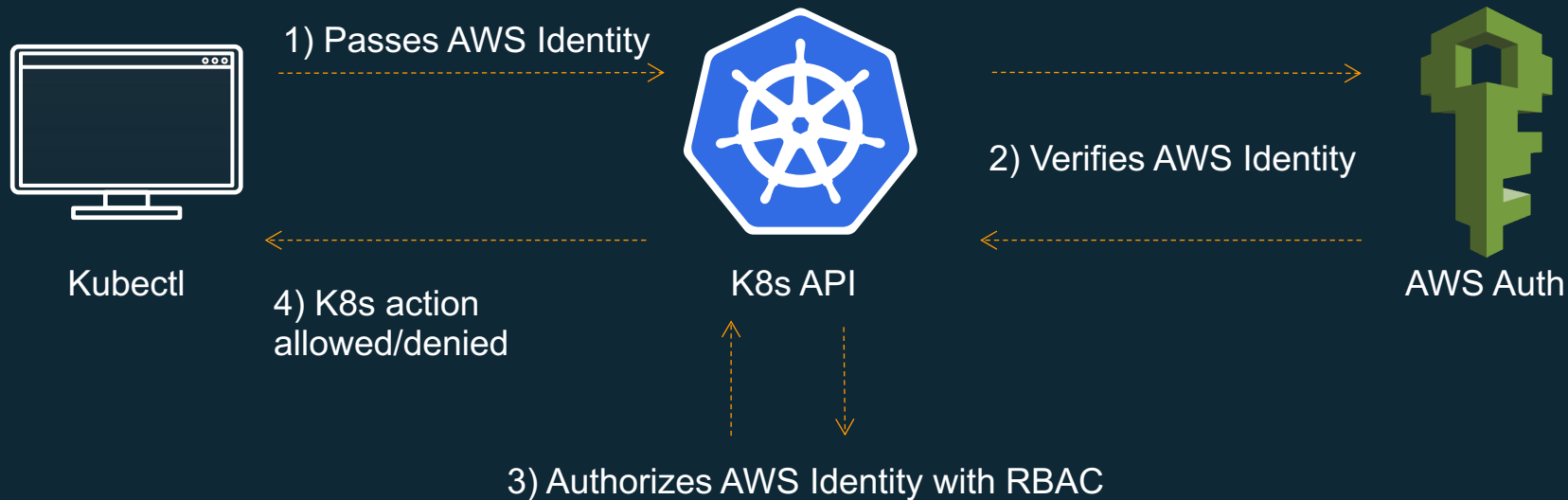
Reference Architecture



<https://eksworkshop.com/spotworkers/>

Authentication & Security

How IAM authentication works with EKS



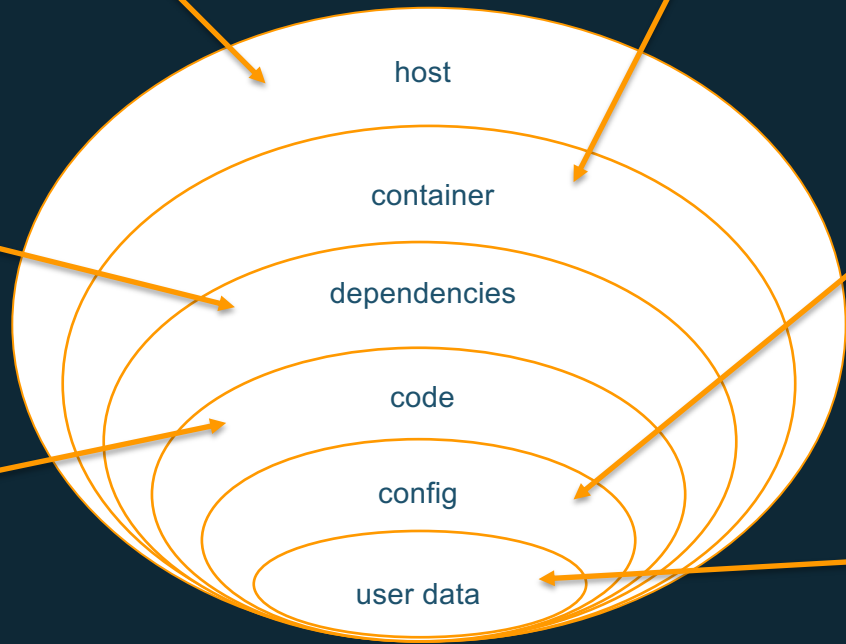
IAM Authentication + Kubectl

- 1) When a Kubectl call is made- IAM identity is passed along with the kubernetes call.
- 2) On the backend, Kubernetes verifies the IAM identity with AWS auth
- 3) The auth response is sent back to Kubernetes, and K8s checks it's internal RBAC mapping for authz. This determines if API call was allowed or denied.
- 4) The K8s API approves or denies the request.

Container security onion model: defense in depth

- full blown distro (Ubuntu, AL) vs. minimal environment (container-optimized distribution)
- multi-tenancy requirements
- **gotchas**: Linux packages/CVEs, leaks, GDPR (in Europe)

- runtime/standards (OCI)
- immutability of images
- all containers share a kernel (mitigation: Firecracker)
- **gotchas**: unnecessary privileged users, no scans, trust



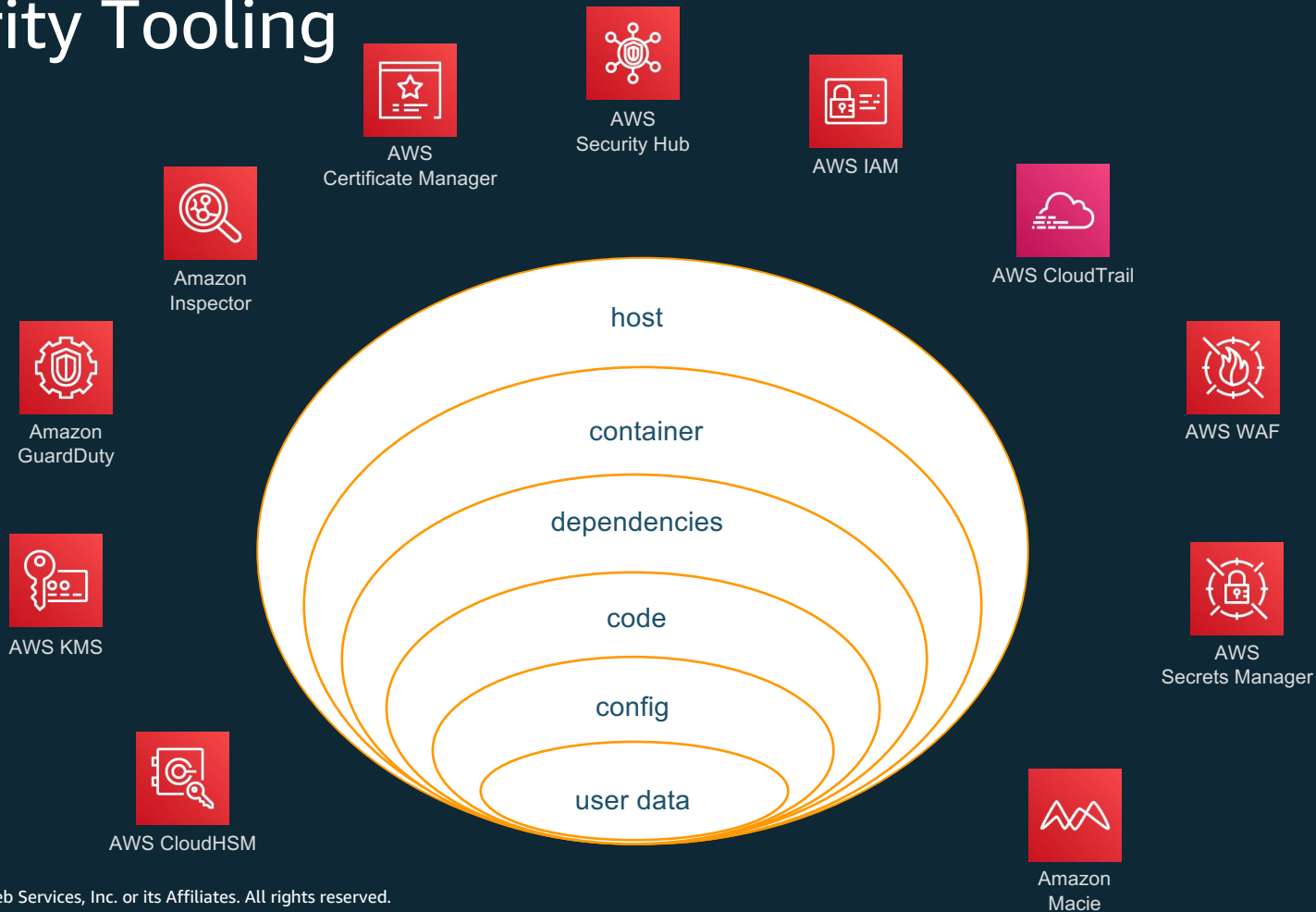
- code analysis
- source available?
- **gotchas**: big surface, many languages

- sensitive config (passwords, API keys, etc.)
- **gotchas**: commits-to-source, non-separated access (dev has cleartext password)

- sanitizing user input
- static code analysis
- **gotchas**: log-leaking

- business core data
- Personal Identifiable Information (PII)
- **gotchas**: leaks, GDPR (in Europe)

Security Tooling



k8s Best Practices

Health Checks

Readiness and liveness probes can help maintain the health of applications running inside Kubernetes. By default, Kubernetes only knows whether or not a process is running, not if it's healthy.

Readiness probes are designed to ensure that an application has reached a "ready" state. In many cases there is a period of time between when a webserver process starts and when it is ready to receive traffic. A readiness probe can ensure the traffic is not sent to a pod until it is actually ready to receive traffic.

Liveness probes are designed to ensure that an application stays in a healthy state. When a liveness probe fails, the pod will be restarted.

Optimizing your container

- Optimize for smaller size, use a multistage Docker build to reduce the size of the runtime container.
- Use a minimalist operating system: Alpine Linux, or similar. Or use no operating system: statically linked Go binary.
- Popular base images have a huge range by size:
node:latest 674MB | node:slim 184MB
java:latest 643MB
ubuntu:latest 85.8MB | alpine:latest 4.41MB | busybox:latest 1.15MB

Configuring resource requests and limits

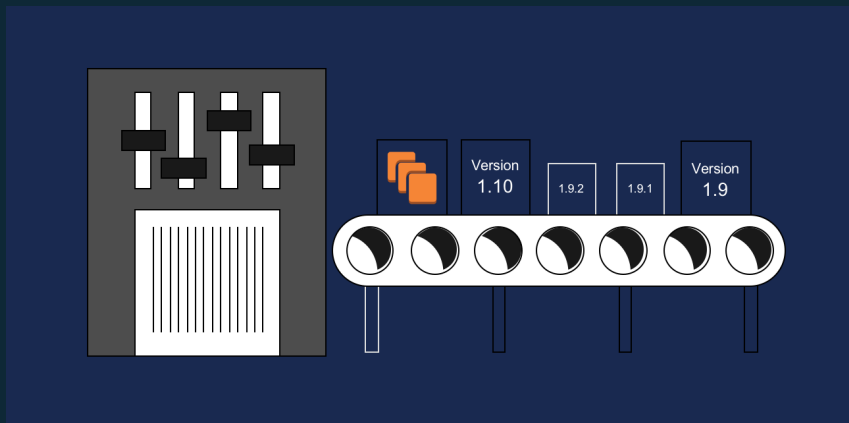
- Setting appropriate resource requests will ensure that all your applications have sufficient compute resources.
- Setting appropriate resource limits will ensure that your applications do not consume too many resources.
- Admission controllers make it easy to add a lot of sidecars but don't underestimate the overhead cost.

```
apiVersion: v1
kind: Pod
metadata:
  name: app
spec:
  containers:
  - name: app
    image: nathanpeck/app
    resources:
      requests:
        memory: "256Mi"
        cpu: "250m"
      limits:
        memory: "512Mi"
        cpu: "500m"
```

When not configured / not correctly configured

- Each container has sufficient access to compute resources. Without resource requests, a pod may be scheduled on a node that is already overutilized.
- Without resource limits, a single poorly behaving pod could utilize the majority of resources on a node, significantly impacting the performance of other pods on the same node.
- Having these values appropriately configured ensures that Cluster autoscaling can function as intended. New nodes are scheduled once pods are unable to be scheduled on an existing node due to insufficient resources. This cannot happen if the resource requests are not configured.

Kubernetes Upgrades



- The k8s version scheme is major.minor.patch, so 1.9.2
- We'll make sure customers have the latest security patches on your cluster, while giving customers a choice of minor versions to run.

Migration between versions

Check if the version can be upgraded

<https://kubernetes.io/docs/setup/release/version-skew-policy/>

Test in a Dev environment if your application is working as expected in the new version

Design a migration strategy

- With minimal downtime
- With no downtime

Container Security Best Practices

Use trusted base images

Use non-root user inside container

Make the file system read only

One process per container

Don't restart on failure, crash cleanly instead

Log to stdout & stderr

Ensure That Images Are Free of Vulnerabilities

Perform automated CVE scans (on push and/or periodically)

Use private registries (ECR, for example)

Ensure That Only Authorized Images are used in your environment

Limit Direct Access to Kubernetes Nodes

Create Administrative Boundaries between Resources

Container Best Practices

Try to aim for reproducible builds (pin dependencies)

Define Resource Quota

Define liveness and readiness probes, always

Implement Network Segmentation

Apply Security Context to Your Pods and Containers

Integrate Security into your CI/CD pipeline

Regularly Apply Security Updates to Your Environment

Make sure you only push approved images to these registries

<https://kubernetes-security.info/>

<https://kubernetes.io/docs/tasks/configure-pod-container/security-context/>

EKS Managed Node Groups

Amazon EKS Managed Node Groups



Nodes with a single command

Use EKS Console, APIs, eksctl, Cloudformation or Terraform.

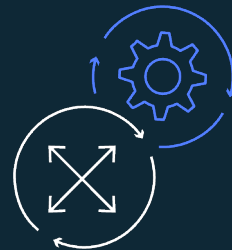
Add Kubernetes labels directly to nodes.



Always up to date

Launch with the latest EKS-optimized AMIs.

Get the latest updates with a single command and rolling updates.



High availability included

Run across multiple availability zones

Tagged for cluster autoscaler discovery

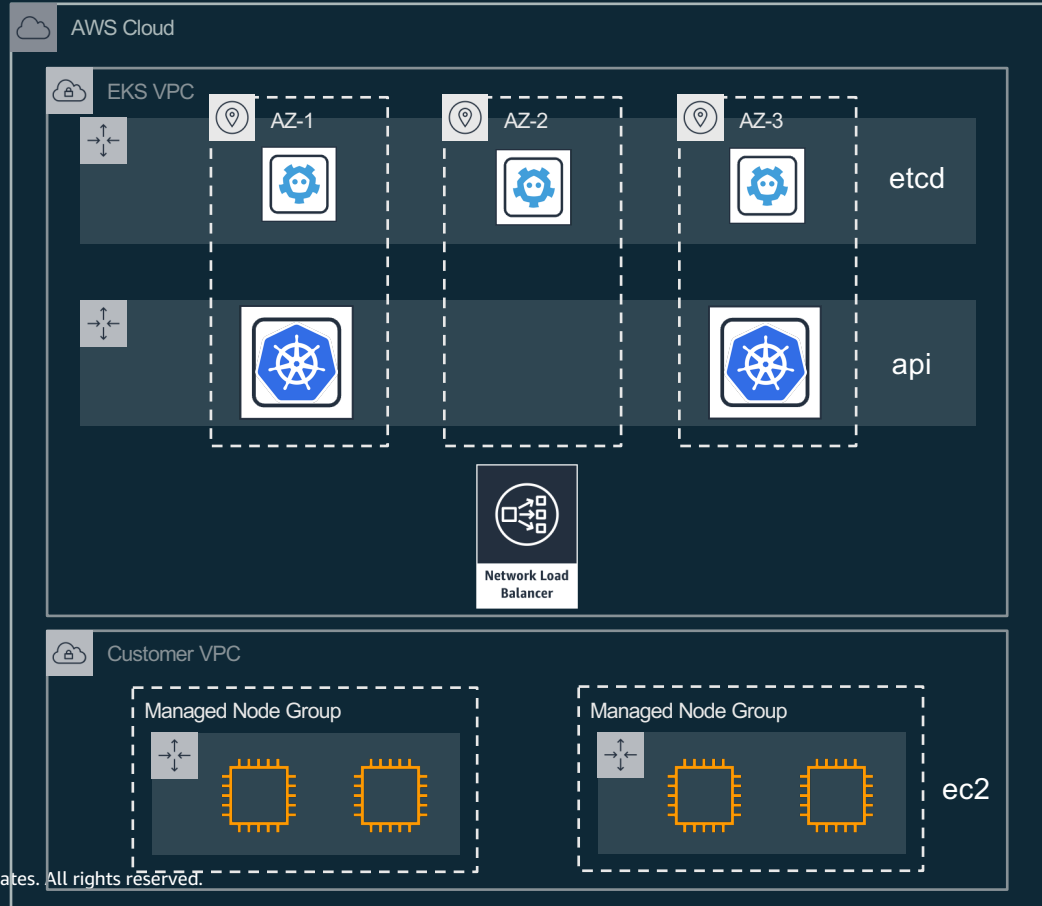
Integrated configuration health checks

Nodes automatically cordon and drain

EKS Cluster Architecture

EKS Managed
Control Plane

EKS Managed
Data Plane



Pricing & Availability

No extra cost to use managed node groups.

Pay standard EC2 prices for any EC2 infrastructure you provision using managed node groups.

Available today for all new 1.14 and above clusters

Use managed node groups in all global EKS regions

Amazon EKS on Fargate

Amazon EKS on Fargate



Bring existing pods

You don't need to change your existing pods.

Fargate works with existing workflows and services that run on Kubernetes.



Production ready

Launch pods quickly. Easily run pods across multiple AZs for high availability.

Each pod runs in an isolated compute environment.



Rightsized and integrated

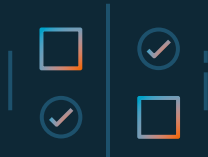
Only pay for the resources you need to run your pods.

Includes native AWS integrations for networking and security.

What matters for Fargate

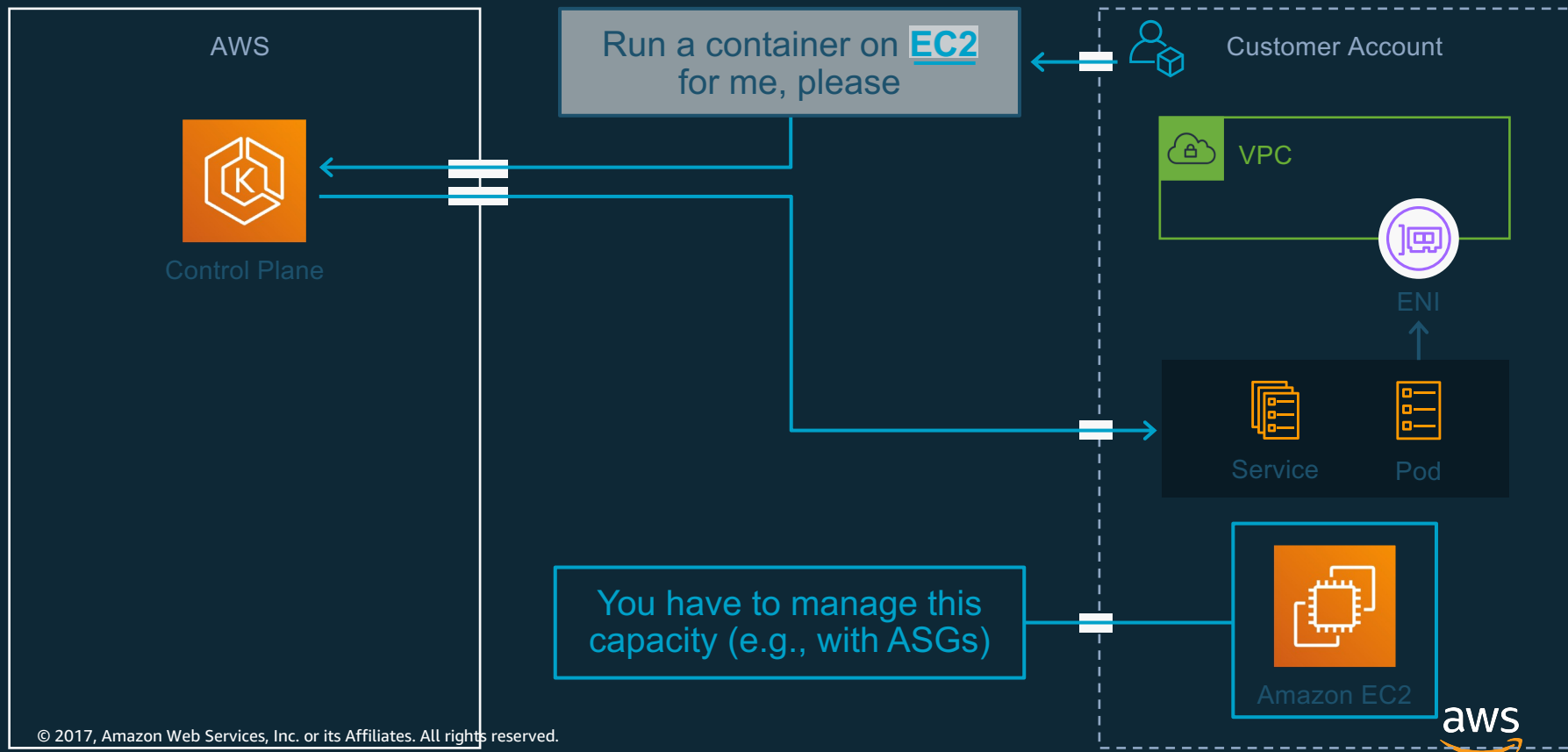


Fargate is a serverless compute platform for containers on AWS

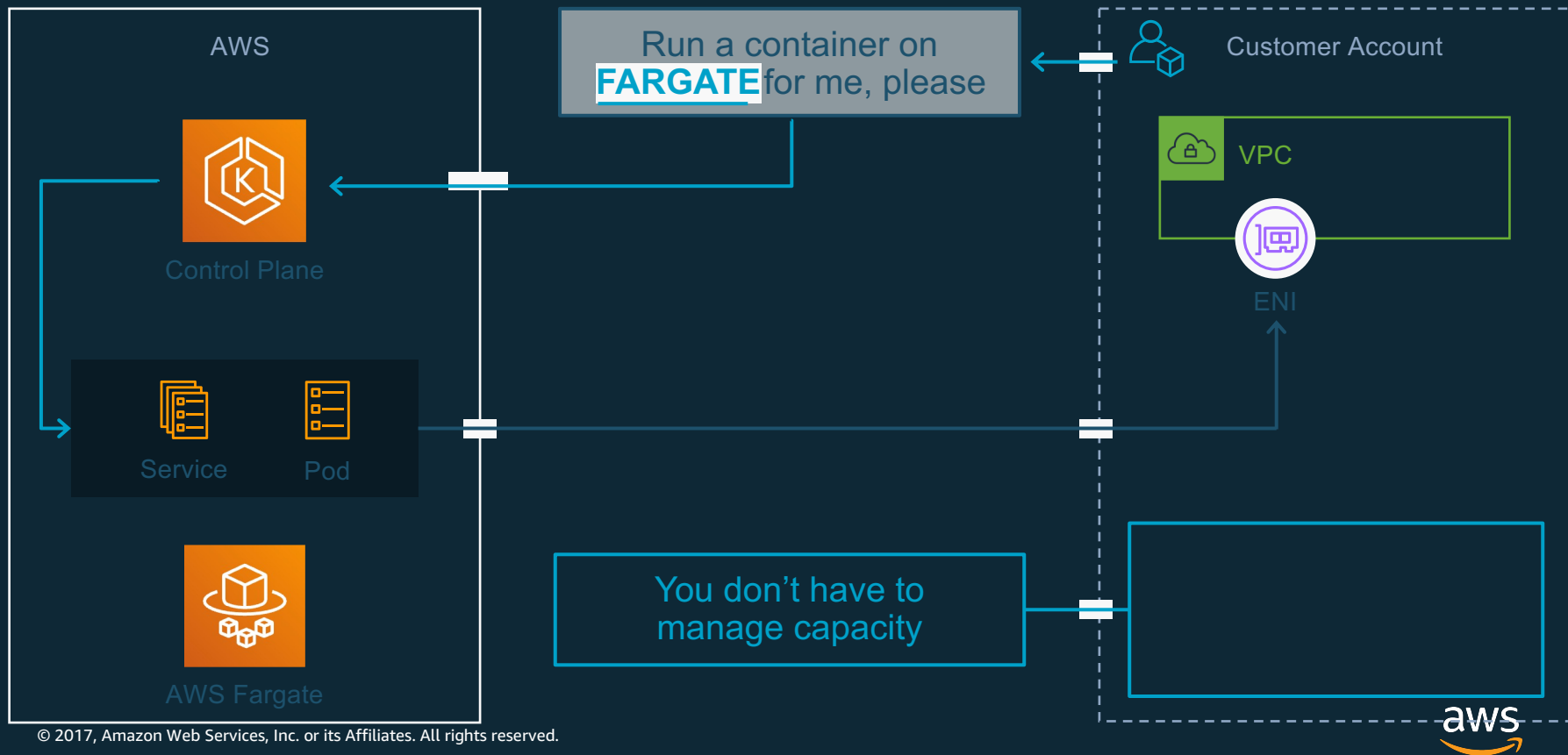


The differences between using EKS and ECS with Fargate are driven by the orchestration system

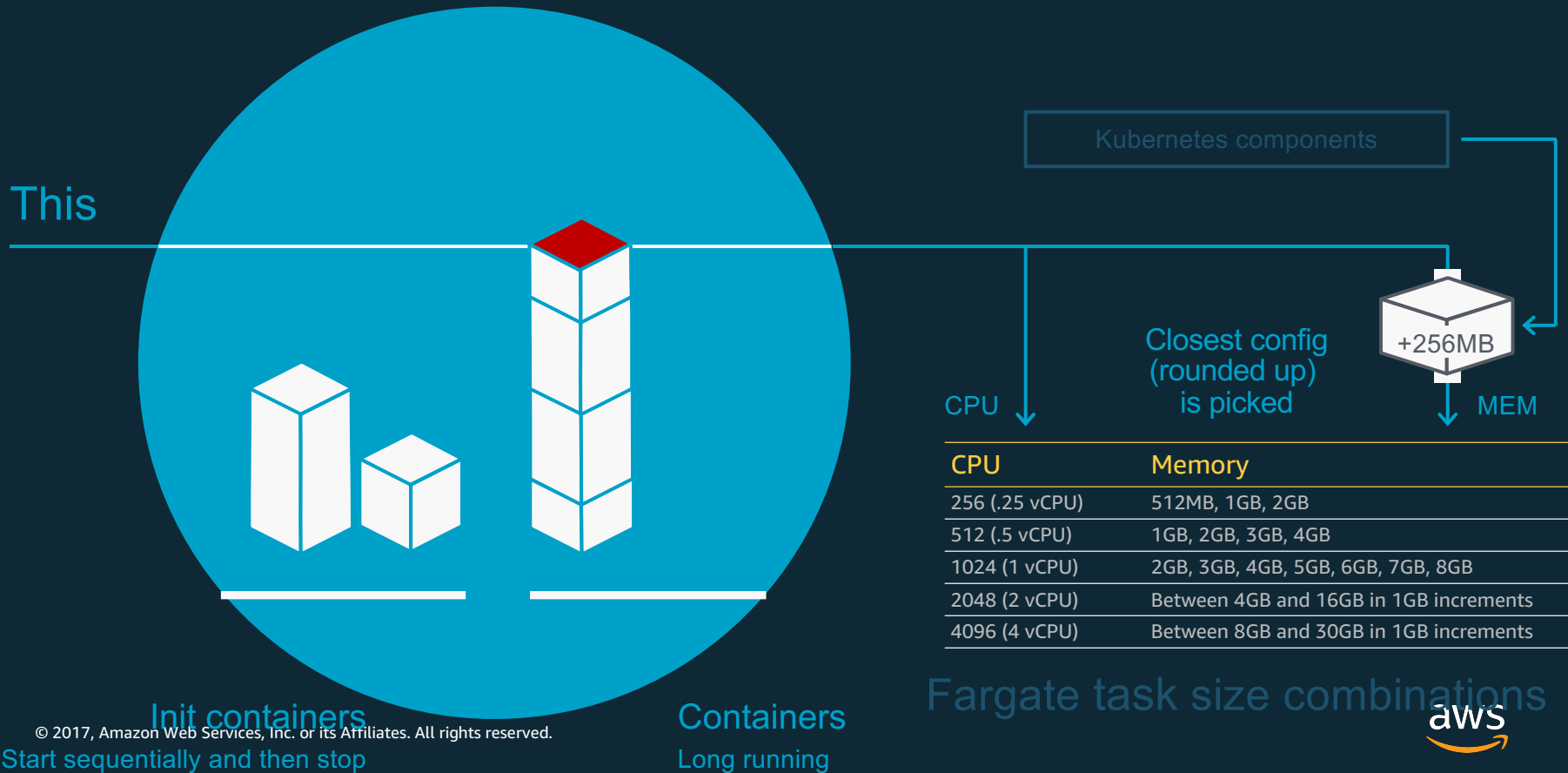
How EKS works on EC2



How EKS works on Fargate



How do we pick the size of the pod?



What is Fargate Spot? (In roadmap for EKS)

- New deployment option for AWS Fargate
- Offers up to 70% discount compared to 'regular' Fargate prices
- Suitable for interruption tolerant workloads
- Simplicity of containers, cost effectiveness of Spot

What are the key benefits of Fargate Spot?

1. Fully managed – no provisioning, patching or diversifying clusters of instances to optimize for pricing option
2. Application first controls - service owners can choose the model that suits their service the best
3. Seamless migration – migrate services with little effort
4. Flexible portfolio of pricing options – customers can choose from regular, Savings Plan or Spot pricing options

Public Roadmaps

The screenshot shows the GitHub repository for the AWS Containers Roadmap. The repository is titled "aws / containers-roadmap" and has 312 Unwatched items, 1,176 Stars, and 24 Forks. The main content area displays a Kanban board with five columns: "We're Working On It", "Coming Soon", "Developer Preview", "Just Shipped", and "Proposed". Each column contains several cards representing different roadmap items, such as "[EKS] CloudFormation support for control plane logging and endpoint access control", "[EKS] [request]: Release CNI Plugin 1.4", "EKS Windows Nodes (preview)", "EKS Control Plane Logs", and "Amazon EFS CSI Driver for Kubernetes". Each card includes a title, a description, a status label (e.g., "EKS", "Proposed", "Developer Preview"), and a link to the issue.

containers-roadmap
Updated 4 days ago

Filter cards

+ Add cards (2 new) Fullscreen Menu

29 We're Working On It

- [EKS] CloudFormation support for control plane logging and endpoint access control
#242 opened by tabern
EKS
- [ECS][Fargate] Service and TaskSet Resource Permissions
#233 opened by coultn
ECS Fargate
- EKS-Optimized AMI Metadata SSM Parameter
#231 opened by tabern
EKS
- [EKS] [request]: EKS Support for Kubernetes 1.14
#212 opened by whereisaaron
EKS Proposed
- New EKS Region : Beijing
#219 opened by wholroyd
EKS Proposed
- DNS resolution for EKS Private Endpoints
#221 opened by tabern
EKS

9 Coming Soon

- [EKS] [request]: Release CNI Plugin 1.4
#149 opened by mogren
EKS
- EKS Support for Kubernetes 1.13
#30 opened by uprightvinyl
EKS
- Control Plane Metrics Endpoint
#182 opened by tabern
EKS
- Support for Public IP space in VPC with EKS
#181 opened by tabern
EKS
- [ECS] [CloudFormation]: CloudFormation support for Secrets
#97 opened by talawahtech
ECS Proposed
- [Fargate, ECS] [CloudFormation]: Support for tags in CloudFormation
#93 opened by iconara
ECS Fargate Proposed
- Fargate IAM Driver Support v1

1 Developer Preview

- EKS Windows Nodes (preview)
#69 opened by ofiliz
Developer Preview EKS

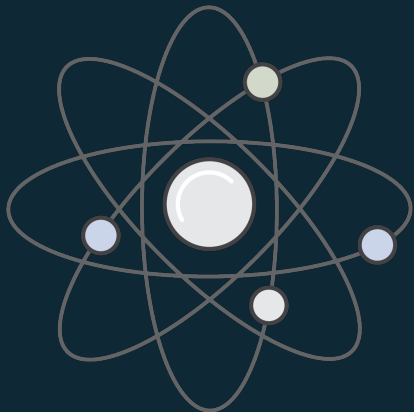
39 Just Shipped

- EKS Control Plane Logs
#26 opened by pauncejones
EKS
- Amazon EFS CSI Driver for Kubernetes
#147 opened by leakingtapan
EKS
- Amazon FSx for Lustre CSI Driver for Kubernetes
#148 opened by leakingtapan
EKS
- [EKS] [request]: EKS optimized AMI for GPU support for CUDA10
#210 opened by mbelang
EKS Proposed
- [EKS] CloudFormation support for cluster upgrades
#115 opened by dcherman
EKS Proposed
- Kubernetes Version 1.11.8
#188 opened by tabern
EKS
- EKS Support for Kubernetes 1.12

<https://github.com/aws/containers-roadmap/projects/1>

Amazon Elastic Container Registry

Amazon Elastic Container Registry



Fully Managed

- Tight Integration with Amazon EC2
- Integration with Docker Toolset
- Management Console and AWS CLI



Highly Available

- Amazon S3 backed
- Regional endpoints



Secure

- IAM Resource-based Policies
- AWS CloudTrail Audit Logs
- Images encrypted at transit and at rest

Access Control

`"ecr:PutImage",
"ecr:InitiateLayerUpload",
"ecr:UploadLayerPart",
"ecr:CompleteLayerUpload",
"ecr:GetAuthorizationToken"`

`"ecr:BatchCheckLayerAvailability",
"ecr:BatchGetImage",
"ecr:GetDownloadUrlForLayer",
"ecr:GetAuthorizationToken"`



Image Scanning

- Automated vulnerability assessment
- Scans for broad range of OS vulnerabilities
- Containers images scanned on push
- Enabled/disabled on a per-repo basis

```
"aws ecr put-image-scanning-configuration --repository-name name --image-scanning-configuration scanOnPush=true --region us-east-2"
```

Image Scanning

- Images can be scanned manually

```
"aws ecr start-image-scan --repository-name name --image-id  
imageTag=tag_name --region us-east-2"
```

- Results available in Console or via CLI

```
"aws ecr describe-image-scan-findings --repository-name name --  
image-id imageTag=tag_name --region us-east-2"
```

- \$0! – 1 per 24hr

Q and A

Thank you!